

André Filipe Siqueira Tokumoto

**Desenvolvimento de um Sistema Web para
Submissão, Avaliação e Publicação de Resumos
Estruturados de Projetos de Extensão e
Pesquisa**

São José dos Campos, Brasil

2026

André Filipe Siqueira Tokumoto

**Desenvolvimento de um Sistema Web para Submissão,
Avaliação e Publicação de Resumos Estruturados de
Projetos de Extensão e Pesquisa**

Trabalho de conclusão de curso apresentado
ao Instituto de Ciência e Tecnologia (UNI-
FESP), como parte das atividades para ob-
tenção do título de Bacharel em Engenharia
de Computação.

Universidade Federal de São Paulo UNIFESP

Instituto de Ciência e Tecnologia

Bacharelado em Engenharia de Computação

Orientador: Prof^a. Dra. Denise Stringhini

São José dos Campos, Brasil

2026

André Filipe Siqueira Tokumoto

Desenvolvimento de um Sistema Web para Submissão, Avaliação e Publicação de Resumos Estruturados de Projetos de Extensão e Pesquisa / André Filipe Siqueira Tokumoto. – São José dos Campos, Brasil, 2026-

67 p. : il. (algumas color.) ; 30 cm.

Orientador: Prof^a. Dra. Denise Stringhini

Trabalho de Graduação – Universidade Federal de São Paulo UNIFESP
Instituto de Ciência de Tecnologia

Bacharelado em Engenharia de Computação, 2026.

1. PEP ICT. 2. Django. 3. Aplicação Web. 4. LaTeX

André Filipe Siqueira Tokumoto

Desenvolvimento de um Sistema Web para Submissão, Avaliação e Publicação de Resumos Estruturados de Projetos de Extensão e Pesquisa

Trabalho de conclusão de curso apresentado
ao Instituto de Ciência e Tecnologia (UNI-
FESP), como parte das atividades para ob-
tenção do título de Bacharel em Engenharia
de Computação.

Trabalho aprovado. São José dos Campos, Brasil, 14 de Julho de 2026:

Prof^ª. Dra. Denise Stringhini
Orientadora

Prof^ª. Dra. Daniela Leal Musa
Convidado 1

Prof. Dr. Tiago de Oliveira
Convidado 2

São José dos Campos, Brasil
2026

Dedico este trabalho aos meus pais, Aparecido e Terezinha, cujo amor incondicional e apoio constante moldaram a pessoa que sou hoje. A gratidão que sinto por vocês é imensurável. Cada página deste trabalho é um reflexo do exemplo de dedicação e valores que aprendi com vocês. Quero também expressar minha profunda gratidão à minha querida irmã Ana, cujo apoio e amizade têm sido um pilar essencial em minha vida. Obrigado por serem minha inspiração e por estarem ao meu lado em todas as jornadas

Agradecimentos

Agradeço aos meus pais, Aparecido e Terezinha, pelo apoio incondicional e pelo suporte fundamental em toda a minha trajetória acadêmica. À minha irmã, Ana, cujo incentivo e palavras de ânimo foram essenciais nos momentos de maior dificuldade. Aos estimados amigos e colegas, pelo companheirismo e apoio mútuo ao longo desta jornada. Aos meus professores, manifesto meu profundo respeito e gratidão pela orientação e ensinamentos.

A educação, qualquer que seja ela, é sempre uma teoria do conhecimento posta em prática” (Paulo Freire)

Resumo

Com a orientação do Ministério da Educação para a curricularização da extensão universitária e a consequente criação dos Programas de Extensão e Pesquisa do ICT (PEPICT), surge a necessidade de padronizar a captação e a divulgação dos resultados desses projetos. Este trabalho propõe a criação de uma aplicação web para ajudar na gestão dos projetos, na qual os alunos possam enviar seus resultados, os professores possam avaliar as entregas e o coordenador possa reunir esses dados em uma publicação organizada e centralizada. O sistema foi construído utilizando o *framework* Django com o banco de dados SQLite, e teve seu desenvolvimento organizado por meio da metodologia ágil Kanban. Uma das principais inovações do sistema é a integração com o compilador L^AT_EX, que permite a geração automatizada de resumos estruturados individuais em PDF e a criação rápida do Caderno de Resumos semestral por parte do coordenador. O software foi desenvolvido para três perfis de usuários: alunos, que ganham autonomia para se inscrever em turmas por códigos de acesso e enviar seus trabalhos; professores, que podem avaliar e enviar pareceres de forma organizada; e coordenadores, que conseguem reunir os trabalhos em uma publicação organizada. O protótipo desenvolvido cumpre todos os requisitos propostos, eliminando a falta de padrão na captação de resultados e criando uma base eficiente para a gestão e a divulgação da extensão universitária no ICT.

Palavras-chaves: PEPICT. Django. Aplicação Web. Automação de Processos. LaTeX.

Abstract

With the Ministry of Education's guidelines for incorporating university extension into the curriculum and the subsequent creation of the ICT Extension and Research Programs (PEPICT), there is a growing need to standardize the collection and dissemination of these projects' results. This work proposes the creation of a web application to assist in project management, where students can submit their results, professors can evaluate the submissions, and the coordinator can gather this data into an organized and centralized publication. The system was built using the *framework* Django with the SQLite database, and its development was organized through the Kanban agile methodology. One of the main innovations of the system is its integration with the L^AT_EX compiler, which enables the automated generation of individual structured abstracts in PDF and the rapid creation of the bi-annual Book of Abstracts by the coordinator. The software was developed for three user profiles: students, who gained autonomy to enroll in classes via access codes and submit their work; professors, who were able to evaluate and send feedback in an organized manner; and coordinators, who managed to compile the approved publications with just a few clicks. The developed prototype fulfills all proposed requirements, eliminating the lack of standardization in data collection and creating an efficient foundation for the management and dissemination of university extension at ICT.

Key-words: PEPICT. Django. Web Application. Process Automation. L^AT_EX.

Lista de ilustrações

Figura 1 – Modelo de resumo estruturado definido para o projeto.	29
Figura 2 – Modelo de revista definido para o projeto.	30
Figura 3 – Modelo de dados.	30
Figura 4 – Diagrama de Blocos.	31
Figura 5 – Diagrama de caso de uso.	32
Figura 6 – Fluxograma do resumo para o Aluno.	33
Figura 7 – Fluxograma de avaliação de resumos para o professor.	33
Figura 8 – Fluxograma do Caderno de Resumos para o coordenador.	33
Figura 9 – Diagrama de sequência de aluno.	34
Figura 10 – Diagrama de sequência de Professor.	35
Figura 11 – Diagrama de sequência de Coordenador.	36
Figura 12 – Controle das etapas com kanban.	37
Figura 13 – Bibliotecas de banco de dados importados	37
Figura 14 – Tabelas criadas durante o desenvolvimento	38
Figura 15 – verificação das credenciais e redirecionamento por tipo de usuário	39
Figura 16 – Geração de chave de acesso aleatória para nova turma	39
Figura 17 – Rotina de busca e validação de turma pela chave de acesso	40
Figura 18 – Validação do status de avaliação do resumo	40
Figura 19 – Seção do resumo com valores padrão	42
Figura 20 – Função de criação do <i>.tex</i> de resumo	42
Figura 21 – Trecho do arquivo texto criado na compilação do caderno de resumos	43
Figura 22 – Compilação do PDF	44
Figura 23 – Chamada de função e retorno do arquivo gerado	44
Figura 24 – tela de <i>login</i>	46
Figura 25 – tela de cadastro de usuário	46
Figura 26 – Painel de Aluno	47
Figura 27 – Tela de inscrição em turma	47
Figura 28 – Tela de edição de Resumo - parte 1	48
Figura 29 – Tela de edição de Resumo - parte 2	48
Figura 30 – Tela de edição de Resumo com trabalho submetido	49
Figura 31 – Painel de Professor	49
Figura 32 – Tela de criação de nova turma	50
Figura 33 – Tela de painel da turma	50
Figura 34 – Tela de análise do resumo - parte 1	51
Figura 35 – Tela de análise do resumo - parte 2	51
Figura 36 – Tela de painel do Coordenador	52

Figura 37 – Tela de painel de turma vigentes	52
Figura 38 – Tela de painel de resumos aprovados da turma	53
Figura 39 – Tela de painel de gerenciamento de tipo de usuário	53
Figura 40 – PDF do resumo gerado pelo sistema	54
Figura 41 – Capa do Caderno de Resumos	55
Figura 42 – Contracapa do Caderno de Resumos	56
Figura 43 – Contracapa do Caderno de Resumos	57
Figura 44 – Introdução do Caderno de Resumos	58
Figura 45 – Resumos componentes da publicação	59
Figura 46 – Sistema Hospedado	60

Lista de tabelas

Tabela 1 – Comparativo entre Sistemas Correlatos e a Plataforma Proposta	20
--	----

Lista de abreviaturas e siglas

ABNT	Associação Brasileira de Normas Técnicas
abnTeX	ABsurd Norms for TeX
API	Application Programming Interface
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
PEPICT	Programas de Extensão e Pesquisa do ICT
ODS	Objetivos de Desenvolvimento Sustentável

Sumário

	Introdução	16
1	OBJETIVOS	17
1.1	Objetivo Geral	17
1.2	Objetivos Específicos	17
2	FUNDAMENTAÇÃO TEÓRICA	18
2.1	Trabalhos Correlatos	18
2.1.1	<i>Open Journal Systems - OJS</i>	18
2.1.2	<i>EasyChair</i>	18
2.1.3	<i>Janeway</i>	19
2.1.4	<i>Journal and Event Management System - JEMS</i>	19
2.1.5	Quadro comparativo	20
2.2	PEPICTs - Programas de Extensão e Pesquisa do ICT	20
2.3	O $\text{\LaTeX}$	21
2.4	Tecnologias Web	21
2.4.1	O Framework Django	21
2.4.2	HTML: Linguagem de Marcação de Hipertexto	22
2.5	Banco de dados SQLite	22
2.5.1	Características do SQLite	23
3	MATERIAIS E MÉTODOS	24
3.1	Etapas do projeto	24
3.1.1	Revisão da literatura	24
3.1.2	Definição de objetivos	24
3.1.3	Desenvolvimento	24
3.1.3.1	Metodologia de desenvolvimento	24
3.1.3.2	Modelagem do sistema	25
3.1.3.3	Implementação	25
3.1.3.4	Testes	26
3.2	Ferramentas	26
3.2.1	Encapsulamento com Pipenv	26
3.2.2	Compilador PDFLaTeX	26
4	DESENVOLVIMENTO	28
4.1	Motivação e definições iniciais	28

4.2	Definição do modelo de projeto	28
4.2.1	Requisitos do projeto	28
4.2.2	Modelos de resumo e caderno de resumos	29
4.2.3	Modelo de dados	30
4.3	Modelagem do sistema	31
4.3.1	Diagrama de Blocos	31
4.3.2	Diagrama de Caso de uso	32
4.3.3	Diagrama de Fluxo de dados	32
4.3.4	Diagrama de sequência	33
4.4	Criação do projeto em Django e Pipenv	36
4.5	Quadro kanban de controle do desenvolvimento	37
4.6	Banco de dados	37
4.7	Autenticação de usuário	38
4.8	Criação de turma e matrícula em turma por aluno	39
4.8.1	Criação de turma	39
4.8.2	Matrícula em turma por aluno	40
4.9	Formulário de resumo e rotina de submissão para avaliação	40
4.10	Avaliação de submissão e feedback	41
4.11	Visualização de resumos aprovados e inclusão na revista no painel de coordenador	41
4.12	Geração do PDF	41
4.12.1	Manipulação do arquivo .tex para resumo	41
4.12.2	Manipulação do arquivo .tex para revista	42
4.13	Compilação do PDF	43
4.14	Deploy	44
5	RESULTADOS E DISCUSSÃO	46
5.1	Resultados obtidos	46
5.1.1	Login no sistema	46
5.1.2	Cadastro de usuário	46
5.1.3	Perfil de Aluno	47
5.1.3.1	Painel de aluno	47
5.1.3.2	Inscrição em turma	47
5.1.3.3	Edição de Resumo Estruturado	48
5.1.4	Perfil de Professor	49
5.1.4.1	Painel de Professor	49
5.1.4.2	Criação de Turma	50
5.1.4.3	Painel da turma	50
5.1.4.4	Tela de análise do resumo	51
5.1.5	Perfil de coordenador	51

5.1.5.1	Painel de Coordenador	51
5.1.5.2	Painel de Turmas Vigentes	52
5.1.5.3	Painel de Resumos aprovados da turma	52
5.1.5.4	Gerenciar tipo de usuário	53
5.1.6	PDF de Resumo gerado	53
5.1.7	PDF do caderno de resumos	54
5.2	Sistema hospedado	59
5.3	Discussão	60
6	CONCLUSÃO	63
	REFERÊNCIAS	65

Introdução

O Plano Nacional de Educação (PNE 2014-2024) estabelece, como uma de suas estratégias, assegurar que os cursos de graduação reservem, no mínimo, 10% do total de seus créditos curriculares para programas e projetos de extensão universitária, orientando sua ação para áreas de pertinência social (EDUCAÇÃO, 2014).

Nesse contexto, a Universidade Federal de São Paulo (UNIFESP) adequou-se às novas normas federais. Após um amplo trabalho de estudo, discussão e planejamento realizado entre 2015 e 2017 pelas equipes da Pró-Reitoria de Extensão e Cultura (PROEC) e da Pró-Reitoria de Graduação (ProGrad), consolidaram-se as diretrizes para a inserção da extensão nos currículos (NACAGUMA; STOCO; ASSUMPÇÃO, 2021). Como parte dessa estratégia no Campus São José dos Campos, foram instituídos os Programas de Extensão e Pesquisa do ICT (PEPICT), que visam facilitar o processo de curricularização e a gestão de indicadores sociais (NACAGUMA; STOCO; ASSUMPÇÃO, 2021).

Com o grande volume de participantes dos projetos PEPICT, existe um novo desafio: reunir os dados do que foi desenvolvido para que a coordenação possa fazer uma análise mais precisa, além de conseguir divulgar as ações desenvolvidas. Dessa forma, surge a ideia de uma plataforma que centralize essas informações e que seja direcionada para as necessidades específicas do ICT da Unifesp.

Diante desse cenário, este trabalho propõe o desenvolvimento de uma plataforma web utilizando o framework Django para a automação do fluxo de submissão, revisão e gestão do caderno de resumos dos projetos e ações desenvolvidas nas disciplinas vinculadas aos PEPICTs. O sistema visa centralizar a comunicação entre coordenadores e participantes, permitindo ainda a compilação das ações aprovadas em uma publicação semestral, otimizando o processo de divulgação dos resultados acadêmicos do ICT-UNIFESP.

1 OBJETIVOS

1.1 Objetivo Geral

Desenvolver um sistema web para a automação do fluxo de resumos estruturados vinculados aos programas PEP ICT, integrando as funcionalidades de elaboração, fluxo de revisão e publicação de caderno de resumos.

1.2 Objetivos Específicos

1. **Análise e modelagem:** levantar requisitos e definir um modelo de dados e fluxo de trabalho adequado ao contexto PEP ICT.
2. **Projeto e implementação:** projetar e construir a plataforma com Django/SQLite, cobrindo autenticação por perfis, fluxo de submissão/revisão e ferramentas de gestão.
3. **Publicação e compilação automatizada :** viabilizar a geração de cadernos de resumos a partir dos projetos aprovados, integrando LaTeX ao processo.

2 Fundamentação Teórica

2.1 Trabalhos Correlatos

Nesta seção, apresenta-se uma análise de trabalhos e ferramentas correlatas na literatura, voltados ao gerenciamento do ciclo de submissão, avaliação e publicação de produções acadêmicas.

2.1.1 *Open Journal Systems* - OJS

O sistema foi criado em 1998 pelo professor John Willinsky, da Universidade da Colúmbia Britânica, que fundou o projeto *Public Knowledge Project*. O grande objetivo da iniciativa era criar ferramentas gratuitas para melhorar a comunicação científica no mundo todo, o que levou ao lançamento oficial da primeira versão do OJS em 2002, atualmente a Universidade Simon Fraser detém os direitos autorais do trabalho produzido pelo Public Knowledge Project([Public Knowledge Project, 2026b](#)).

O *Open Journal Systems* (OJS) é um *software* de código aberto para a criação e gestão de revistas científicas. Ele permite que universidades e pesquisadores publiquem artigos na internet ao automatizar todo o processo de publicação, que vai desde o envio do texto pelos autores até a revisão e a aprovação final ([Public Knowledge Project, 2026a](#)).

A ideia geral do fluxo de trabalho do projeto desenvolvido segue a mesma lógica do OJS, em que autores enviam seus trabalhos para serem avaliados por revisores selecionados e publicados após a aprovação. Porém, o OJS não adota o padrão de organizar a produção por disciplinas e projetos variados, revisados por diferentes pessoas, mas unificados em uma mesma publicação final.

2.1.2 *EasyChair*

A *EasyChair* é uma plataforma desenvolvida no Reino Unido que tem como objetivo de fazer a gestão de trabalhos apresentados em eventos acadêmicos, dentre seus recursos estão a submissão de resumos, revisão por pares quanto a publicação dos trabalhos revisados nos formatos Word e LaTeX ([EasyChair, 2026](#)).

A plataforma fornece soluções diversas para a gestão de eventos, indo desde o formulário e taxas de inscrição até a publicação dos trabalhos apresentados, inclusive no formato LaTeX ([EasyChair, 2026](#)). Entretanto, não se trata de um serviço gratuito e seu escopo é focado em eventos. Apesar disso, a ferramenta apresenta o uso prático de LaTeX, o que é de grande interesse por ser uma das tecnologias adotadas no presente trabalho.

2.1.3 *Janeway*

O *Janeway* é uma plataforma de código aberto adaptável que fornece os serviços de submissão de manuscritos, revisão por pares e publicação dos trabalhos, desenvolvido em Python e Django sendo uma alternativa mais leve ao *Open Journal Systems* ([Open Library of Humanities, 2026](#)).

A plataforma *Janeway* oferece muitos dos recursos buscados neste trabalho. Porém, ela não fornece uma estrutura dividida em disciplinas com revisores específicos por projeto, nem a lógica de um mesmo autor ter trabalhos publicados em diversas disciplinas. Para atingir esse objetivo, seria necessária a reformulação de boa parte do sistema. Mesmo assim, essa plataforma é a que mais se aproxima do projeto desenvolvido, principalmente pelo uso do Django.

2.1.4 *Journal and Event Management System - JEMS*

O Sistema dedica-se a gerenciar os processos de submissão, revisão e notificação de conferências acadêmicas. O JEMS é mantido pelo Grupo de Redes de Computadores (CNG) da Universidade Federal do Rio Grande do Sul ([Sociedade Brasileira de Computação, 2026a](#)).

O sistema possui uma estrutura de usuários composta por coordenador, autor e revisor. O coordenador define os tópicos e prazos, abrindo o evento para a submissão de trabalhos. Enquanto o período de submissão está aberto, ele configura os formulários e os prazos de revisão. Quando esse prazo se encerra, o coordenador distribui os trabalhos entre os revisores de maneira arbitrária ou automática. Por fim, o sistema devolve uma lista com os trabalhos ordenados por desempenho, servindo de base para o coordenador selecionar os artigos aceitos pelo evento ([Sociedade Brasileira de Computação, 2026b](#)).

O grande problema de tentar usar o JEMS é que ele é muito bom para eventos pontuais. Só que os projetos de extensão exigem um processo que se repete a cada semestre, integrado ao dia a dia das aulas na universidade, além disso trata-se de uma ferramenta paga o que traria custos relevantes para a universidade ([Sociedade Brasileira de Computação, 2026c](#)). Adotar o JEMS significaria abrir um 'evento' novo a cada semestre para cada disciplina. Isso criaria uma grande carga de trabalho manual para a coordenação do PEP ICT, além de deixar os dados dos projetos espalhados.

2.1.5 Quadro comparativo

A seguir é apresentado um quadro comparativo resumindo as diferenças entre os trabalhos analisados e o sistema proposto.

Tabela 1 – Comparativo entre Sistemas Correlatos e a Plataforma Proposta

Funcionalidade	OJS	EasyChair	Janeway	JEMS	Plataforma Proposta (PEPICT)
Licença / Custo	Gratuito	Pago	Gratuito	Pago	Gratuito (Customizado)
Estrutura por Disciplinas	Não	Não	Não	Não	Sim (Inscrição por Código)
Automação LaTeX na Web	Não	Parcial	Não	Não	Sim (Geração Automatizada)
Compilação da Coletânea	Não	Não	Não	Não	Sim (Caderno em 1 clique)

Fonte: O autor.

2.2 PEPICTs - Programas de Extensão e Pesquisa do ICT

Os Programas de Extensão e Pesquisa do ICT (PEPICT) ([NACAGUMA; STOCO; ASSUMPÇÃO, 2021](#)) constituem-se de cursos, pesquisas, serviços, eventos e demais ações realizadas conjuntamente entre o Instituto de Ciência e Tecnologia e a comunidade. Integram atividades de ensino, pesquisa e extensão alinhadas a pelo menos um dos Objetivos de Desenvolvimento Sustentável (ODS) da Organização das Nações Unidas (ONU).

O objetivo desses programas é contribuir no processo de curricularização da extensão, entendida como o processo de incorporação das atividades de extensão universitária no currículo das unidades curriculares dos cursos de graduação e pós-graduação, destacando a ação social das ações extensionistas. Além disso, buscam facilitar a captação de recursos por editais próprios e a participação em editais externos.

A estrutura organizacional segue a seguinte definição:

1. **Coordenador Geral:** Escolhido entre os coordenadores de projeto para um mandato de 2 anos. Tem como objetivo articular a colaboração entre os coordenadores de projeto, além de reunir informações e indicadores relevantes para a melhoria constante do programa .

2. **Coordenador de Projeto:** Responsável por uma ação processual e contínua de caráter educativo, social, cultural, científico ou tecnológico com objetivo específico e prazo determinado .
3. **Participantes dos projetos:** Envolvem o corpo discente (alunos em disciplinas curricularizadas), docentes, servidores e a comunidade externa .

2.3 O L^AT_EX

O L^AT_EX é um editor de texto formulado para entregar uma alta qualidade de imagem em PDF, criado especialmente diante da baixa qualidade das formulas matemáticas em arquivos nos anos 80 , mas a linguagem se expandiu hoje sendo uma linguagem utilizada em varios setores da publicação científica (UFF, 2004).

A linguagem utiliza o processamento de texto no estilo lógico, ou seja, o texto a ser impresso e os comandos de formatação são escritos em um mesmo arquivo, que posteriormente é compilado em um arquivo de saída que pode ser visualizado: como pdf ou html (LATEX PROJECT, 2024).

Justamente por essa qualidade visual, em conjunto com a facilidade de padronização de *layout* do arquivo compilado é o que buscamos com o uso do L^AT_EX nesse trabalho.

2.4 Tecnologias Web

2.4.1 O Framework Django

O *Django* é um *framework* de Python para o desenvolvimento de aplicativos web gratuito e de código aberto. No site oficial do *framework Django*, ele é descrito como uma estrutura da *Web Python* de alto nível que incentiva o desenvolvimento rápido e o design limpo e pragmático (Django Software Foundation, 2026b).

As APIs em Django são estruturadas com a arquitetura *Model View Template* (MVT), que consiste em um *objetic relational mapper* (ORM), que faz a mediação entre os modelos de dados, que em Python são definidos como classes (Django Software Foundation, 2026b), sendo eles:

- **Model :**

Um modelo é a fonte única e definitiva de informações sobre os seus dados. Ele contém os campos e comportamentos essenciais relacionados aos dados que estão sendo armazenados. Geralmente, cada modelo é mapeado para uma tabela específica no banco de dados (Django Software Foundation, 2026c).

- Cada modelo é uma classe Python que herda `django.db.models.Model`.
- Cada atributo de um modelo representa um campo no banco de dados.
- O Django fornece uma API de acesso ao banco de dados que é gerada automaticamente

- **View :**

As funções na classe *View*, definem os procedimentos que a API vai realizar para cada tipo de solicitação HTTP recebida, de acordo com o tipo de requisição (*GET*, *POST*, *DELETE*, etc), é retornado o conteúdo correspondente a solicitação (Django Software Foundation, 2026a).

- **Template :**

O *template* contém as partes estáticas da saída HTML desejada e inclui uma sintaxe especial que descreve como o conteúdo dinâmico será inserido. É nesse ponto que ocorre a renderização dos dados provenientes das *views*, permitindo que o Django apresente as solicitações aos usuários de maneira eficaz. (Django Software Foundation, 2026d)

O Django é uma tecnologia robusta, de fácil aprendizagem e manutenção. Por proporcionar uma ampla gama de possibilidades de desenvolvimento, torna-se uma escolha bastante interessante para projetos web, além de ser a opção ideal para este trabalho.

2.4.2 HTML: Linguagem de Marcação de Hipertexto

O HTML é a estrutura de construção mais básica da web. Define o significado e a estrutura do conteúdo; nele, são definidos os elementos que compõem a página web, como textos, títulos, cabeçalhos, menus, botões e demais elementos que fazem parte de uma aplicação na web. (MDN CONTRIBUTORS, 2026).

Para este projeto, utilizou-se um *front-end* construído apenas com HTML, considerando a limitação de tempo e o maior foco na funcionalidade do sistema. Dessa forma, as páginas estruturadas fornecem os recursos necessários para cumprir esse objetivo.

2.5 Banco de dados SQLite

O **SQLite** é um mecanismo de banco de dados SQL incorporado, o qual não demanda um processo de servidor independente. Seu funcionamento baseia-se na manipulação de dados armazenados diretamente em arquivos de disco comuns (SQLite CONSORSIUM, 2025).

Pode-se entender o SQLite como um banco de dados embarcado, pois, ao invés de rodar como um processo independente, ele simbioticamente coexiste dentro da aplicação onde ele opera, funcionando como parte do programa que ele serve. Com cliente e servidor operando no mesmo processo, o tempo de processamento e acesso ao banco reduz drasticamente (OWENS; ALLEN, 2010).

2.5.1 Características do SQLite

O SQLite foi criado em 2000 pelo programador Dwayne Richard Hipp, com a intenção de ser uma alternativa de banco de dados de fácil implementação, com processamento leve e compatível com diversos sistemas e aplicações (OWENS; ALLEN, 2010).

- **Sem necessidade de configuração**

O SQLite tem, desde sua concepção, a capacidade de ser incorporado e usado sem a necessidade de muita configuração, sendo assim, de simples instalação nas aplicações (OWENS; ALLEN, 2010).

- **Portabilidade**

O SQLite foi desenvolvido já considerando a portabilidade, ele pode ser executado em Windows, Linux, DBS, Mac OS, entre outros. Ele funciona bem tanto em sistemas de 32 bits, quanto em sistemas de 64 bits (OWENS; ALLEN, 2010).

- **Compactação**

O SQLite foi projetado para ser leve e autossuficiente, é um banco de dados completo que pode ser armazenado em um único arquivo de disco multiplataforma (SQLite CONSORSIUM, 2025).

- **Flexibilidade**

Como um banco de dados incorporado, o SQLite oferece o poder e flexibilidade de um front-end de banco de dados relacional e a compactação e simplicidade de um back-end de árvore binária; dessa forma, ele é um suporte SQL simples integrado diretamente ao aplicativo (OWENS; ALLEN, 2010).

O SQLite oferece uma estrutura de banco de dados completa e totalmente integrada, o que dispensa configurações complexas e acelera o ritmo de trabalho. Por ser leve e eficiente, ele se mostrou a escolha perfeita para o desenvolvimento do protótipo deste sistema, permitindo focar no que realmente importa: validar as funcionalidades da aplicação de maneira ágil.

3 Materiais e métodos

Nesse capítulo, são expostas as etapas de planejamento e desenvolvimento desse trabalho, bem como as metodologias empregadas e ferramentas utilizadas.

3.1 Etapas do projeto

3.1.1 Revisão da literatura

Uma vez definida a problemática da despadronização nas entregas de resultados dos PEPICT, foi realizada uma pesquisa por plataformas existentes com objetivos similares. Foram encontradas soluções com premissas parecidas, mas que apresentam diferenças consideráveis em relação ao cenário ideal para a realidade das disciplinas curricularizadas. Entre as limitações, destacam-se a ausência de uma organização por turmas com revisores específicos e a incapacidade de compilar todos os trabalhos em uma única publicação. Por outro lado, as ferramentas que atendem a boa parte desses requisitos são pagas, o que não apenas manteria a sobrecarga de trabalho dos coordenadores, mas também geraria um custo financeiro extra para a universidade.

3.1.2 Definição de objetivos

Para a definição dos objetivos, foram realizadas reuniões com a orientação para investigar quais são as necessidades reais dos usuários potenciais e o que seria esperado de um produto final. Com essa definição, foram estruturados os pontos principais da aplicação e quais as necessidades de cada tipo de usuário envolvido nos PEPICT. Esse mapeamento foi fundamental para desdobrar o objetivo geral deste trabalho em metas específicas, servindo como base para as próximas etapas de desenvolvimento do sistema.

3.1.3 Desenvolvimento

3.1.3.1 Metodologia de desenvolvimento

A metodologia empregada nesse projeto foi de acordo com as metodologias ágil, que é uma abordagem de desenvolvimento que visa criar um fluxo de trabalho para tornar o processo de produção o mais eficiente e assertivo possível (BECK et al., 2001). Em geral essas metodologias são utilizadas para organizar o trabalho de uma equipe de desenvolvimento, mas também pode ser aplicado para organizar as tarefas e cronograma de desenvolvimento em trabalhos individuais.

Foi desenvolvida em 2001, quando seus autores buscaram solucionar lacunas e ineficiências do desenvolvimento em cascata, detectadas quando essa metodologia não produzia mais resultados satisfatórios frente a vários parâmetros de medida. Era comum, por exemplo, demorar anos entre a identificação de uma necessidade da empresa e a entrega do sistema pronto. Nesse meio-tempo, as mudanças no mercado e nas exigências do negócio faziam com que grande parte do projeto fosse cancelada antes mesmo de ser entregue. Esse desperdício de tempo e dinheiro motivou vários desenvolvedores a buscarem uma alternativa. ([RED HAT, 2024](#)).

Nesse contexto surgiram varias metodologias de tornar o processo de produção mais rápido e assertivo, entre essas diferentes abordagens, a que foi utilizada nesse projeto foi o método Kanban que se trata de uma ferramenta visual para gerenciar o fluxo de produção. Ele é representado por um quadro composto por colunas, que refletem as etapas de desenvolvimento, e por cartões, que indicam as tarefas. À medida que uma atividade avança em uma determinada fase, seu cartão é movido para a coluna seguinte; se necessário, a tarefa pode ser recolocada em uma coluna pela qual já tenha passado ([WAKODE; RAUT; TALMALE, 2015](#)). Esse método foi empregado neste projeto para auxiliar na visualização e organização das partes a serem desenvolvidas, no controle do cronograma planejado e no andamento efetivo do processo.

3.1.3.2 Modelagem do sistema

Finalizada a etapa de definição de objetivos, deu-se início à modelagem do sistema para a visualização de suas partes e interações. Esse mapeamento foi concretizado por meio de diagramas de blocos, de sequência, de casos de uso e de um fluxograma. Posteriormente, alinhado a essa especificação, o banco de dados foi modelado na ferramenta brModelo, o que possibilitou estruturar os campos e relacionamentos necessários para guiar as próximas fases do desenvolvimento.

3.1.3.3 Implementação

Com todo o planejamento realizado, foi iniciada a implementação do sistema. Após as etapas de configuração do repositório Git e da aplicação Django, o primeiro elemento desenvolvido foi o banco de dados, pois ele era a base para a realização das etapas essenciais posteriores, como a autenticação de usuários e as restrições de acesso.

Como o *framework* Django adota o padrão *Model-View-Template* (MVT), as etapas de desenvolvimento das partes integrantes do sistema seguiram a lógica de implementação da *view*, criação da URL e desenvolvimento do HTML já interligado à *view* correspondente. Essas partes foram sendo desenvolvidas seguindo também a sequência lógica de acesso do usuário: primeiro o painel de controle, depois as partes acessíveis por esse painel, e assim sucessivamente, o que tornou mais fácil a detecção de problemas durante o desenvolvimento.

3.1.3.4 Testes

Diante das restrições de tempo para a conclusão deste trabalho, não foi realizada uma etapa formal de testes de usabilidade com usuários potenciais. Em contrapartida, focou-se na execução de testes funcionais, realizados pelo próprio desenvolvedor, com o objetivo de garantir a estabilidade das regras de negócio e o correto comportamento da interface antes da disponibilização do sistema.

Esses testes consistiram na validação sistemática de cada fluxo da aplicação. Foram simulados cenários de login com credenciais válidas e inválidas, tentativas de acesso a páginas restritas sem a devida autenticação, e o preenchimento de formulários de envio de resultados dos PEP ICT, verificando se o banco de dados persistia as informações corretamente e se as restrições de perfil de usuário (aluno, professor e coordenador) funcionam conforme o esperado.

Após a validação interna das funcionalidades e a correção das falhas identificadas, o sistema foi preparado para o ambiente de produção. A plataforma foi hospedada com sucesso utilizando o serviço *Fly.IO*, permitindo que a aplicação saísse do ambiente local (*localhost*) e ficasse acessível de forma pública através da internet. O processo de *deploy* foi integrado ao repositório Git, facilitando a atualização contínua do sistema.

3.2 Ferramentas

3.2.1 Encapsulamento com Pipenv

O Pipenv é uma ferramenta de gerenciamento de pacotes que une o *pip* ao *virtualenv*. Ele permite que as dependências e recursos utilizados em um projeto *Python* fiquem isolados, evitando a necessidade de instalá-los globalmente na máquina onde o projeto será desenvolvido ([Python Packaging Authority, 2026](#)).

O uso dessa ferramenta otimiza significativamente o processo de desenvolvimento com *Django*, pois gerencia todas as dependências do projeto de forma organizada. Além disso, ela simplifica a futura hospedagem do sistema em um servidor web, garantindo que a aplicação funcione em produção exatamente da mesma maneira que no ambiente local, evitando conflitos de versões.

3.2.2 Compilador PDFLaTeX

O PDFLaTeX é um módulo que tem como objetivo compilar arquivos com a extensão *.tex* diretamente para o formato PDF, preservando a alta qualidade visual característica do LaTeX ([MBELLO, 2019](#)). Devido à sua facilidade de integração, estabilidade e excelente desempenho no processamento de documentos textuais e elementos gráficos, essa ferramenta

foi a escolha ideal para realizar a geração automatizada dos arquivos PDF dentro do sistema.

O uso do LaTeX por meio desse módulo viabiliza, de forma simplificada, a padronização e a formatação dos trabalhos conforme as regras da ABNT. Além disso, a tecnologia proporciona uma qualidade visual elevada para os documentos finais, disponibilizando à comunidade do campus uma ferramenta de alto desempenho estético. Esse nível de precisão gráfica e uniformidade técnica seria de complexa execução caso fossem utilizadas ferramentas tradicionais de edição de texto.

4 Desenvolvimento

4.1 Motivação e definições iniciais

O projeto foi motivado pela falta de padronização nos resultados dos PEPICTs desenvolvidos nas Unidades Curriculares e projetos de extensão do ICT-Unifesp. Atualmente, a necessidade de cada coordenador criar seus próprios critérios de entrega prejudica a eficiência da avaliação exigida pelo PNE. Buscamos centralizar esse processo para torná-lo mais ágil e preciso.

Após mapear as melhores formas de viabilizar o projeto, ficou definido que a solução ideal seria uma aplicação web. O sistema conta com uma área de login para que cada usuário acesse seus dados específicos e gere seu relatório individual em PDF. Para a gestão geral, o perfil de administrador tem a autonomia de consolidar os trabalhos do semestre letivo em uma única coletânea.

4.2 Definição do modelo de projeto

4.2.1 Requisitos do projeto

Para garantir que o sistema atenda perfeitamente à demanda de padronização, a etapa de levantamento de requisitos foi realizada de forma conjunta com a orientação do projeto. Por meio de reuniões de análise, foi possível definir o escopo da ferramenta e o papel de cada usuário no fluxo de entrega dos relatórios. Os principais requisitos identificados e validados para o desenvolvimento da aplicação web estão apresentados a seguir:

1. O sistema deve disponibilizar três níveis de permissão de acesso: aluno, professor e coordenador.
2. O sistema deve permitir que o aluno realize cadastro no sistema, vincule-se a turmas, edite suas informações, envie trabalhos para avaliação e exporte seu resumo em PDF.
3. O sistema deve permitir que o professor crie turmas, acesse os trabalhos enviados, realize avaliações, retorne *feedbacks* e faça o download dos arquivos dos estudantes.
4. O sistema deve permitir que o coordenador filtre turmas por semestre, visualize projetos aprovados, selecione os trabalhos que farão parte do caderno de resumos e gere o arquivo PDF consolidado da edição.

5. Para melhor qualidade do PDF e padronização conforme as regras da ABNT a geração do PDF se dará por intermédio do LaTeX.

4.2.2 Modelos de resumo e caderno de resumos

Concluída a etapa de requisitos, o passo seguinte foi estruturar o modelo de resumo. Para isso, realizamos reuniões com a orientação do projeto para alinhar e selecionar quais informações seriam essenciais e pertinentes para compor esses resumos estruturados, o que resultou no modelo ilustrado na imagem abaixo:

**Modelo de Resumo Expandido – Caderno de Resumos PEP ICT –
ICT/Unifesp**

Preencha os campos abaixo com as informações referentes ao trabalho desenvolvido em sua disciplina com componente de extensão vinculada aos Programas PEP ICT.

Título do Trabalho

Autores(as)
Nome completo dos(as) estudantes e docente(s) responsável(eis)

Disciplina e Curso
Nome da UC curricularizada, curso(s) e semestre

Programa PEP ICT vinculado
☐ Educação e Sustentabilidade
☐ Tecnologia e Inovação para o Desenvolvimento Social

Objetivos do Trabalho
Descreva o objetivo geral da atividade e o contexto/motivação.

Metodologia
Explique as etapas do trabalho e como se deu a interação com a comunidade (ou justifique a proposta, caso não tenha interação).

Resultados e Produtos
Liste os produtos gerados (protótipos, oficinas, relatórios, etc.) e os resultados alcançados.

Relação com os ODS
Indique até 3 ODS trabalhados e justifique como o trabalho contribuiu com cada um deles.

Reflexão crítica dos estudantes
Relate os principais aprendizados, desafios e percepções sobre a extensão universitária.

Referências (se houver)
Inclua as fontes bibliográficas utilizadas (formato ABNT).

Figura 1 – Modelo de resumo estruturado definido para o projeto.
Fonte: Dados fornecidos pela orientadora (2026).

Após a definição do modelo de resumo foi definido o modelo de revista, com uma estrutura de dados introdutórios e resumos organizados por projeto PEP ICT, seguindo o seguinte modelo (Figura 1):

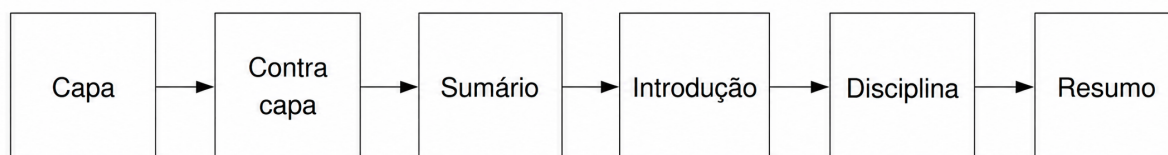


Figura 2 – Modelo de revista definido para o projeto.

Fonte: O autor

4.2.3 Modelo de dados

Com os modelos de relatório e revista estabelecidos, a etapa seguinte consistiu na modelagem do banco de dados. O projeto do esquema considerou as informações essenciais para os cadastros de usuários e turmas, além dos dados estruturais dos resumos e relatórios, mapeando com precisão os relacionamentos entre as tabelas.

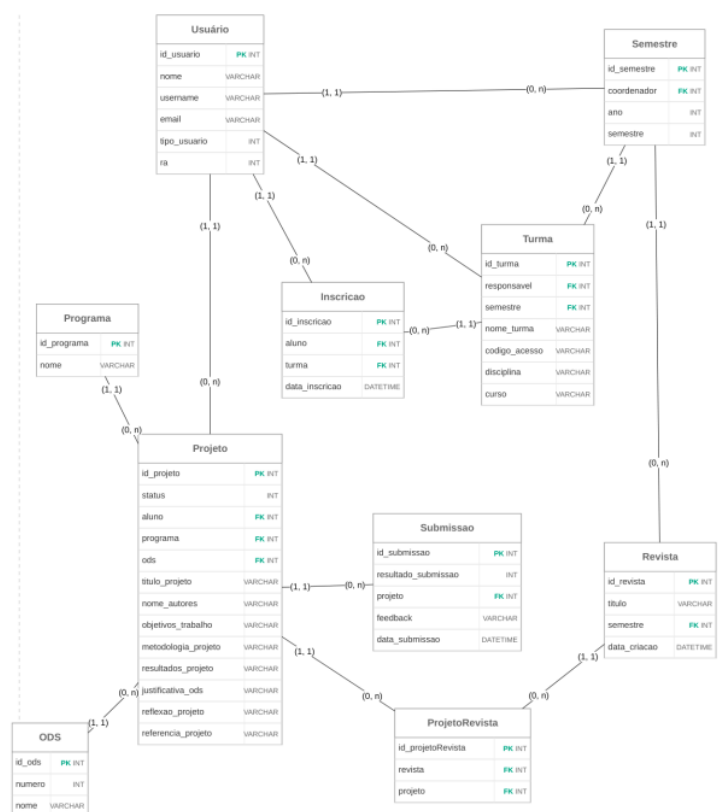


Figura 3 – Modelo de dados.

Fonte: O autor

4.3 Modelagem do sistema

Com os requisitos alinhados e a modelagem inicial do banco de dados pronta, o passo seguinte foi desenhar os diagramas do sistema. Essa etapa funcionou como um mapa para planejar as tarefas e trouxe uma visão muito mais clara de como cada funcionalidade deveria ser construída.

4.3.1 Diagrama de Blocos

O primeiro diagrama modelado foi o diagrama de blocos, ele permite a visualização das estruturas do sistema e sua distribuição (BOOCH; RUMBAUGH; JACOBSON, 2006).

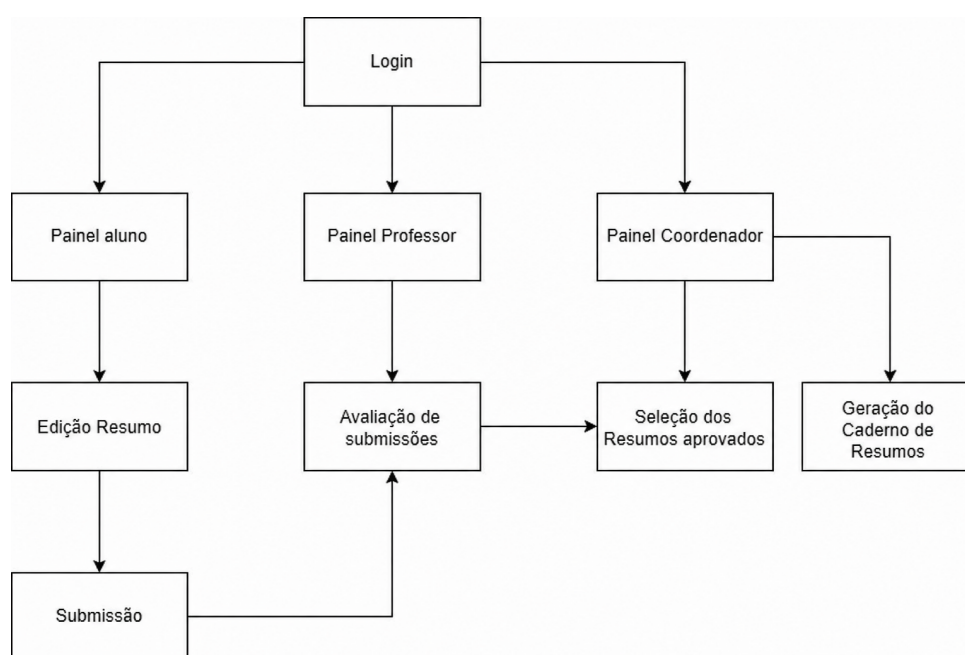


Figura 4 – Diagrama de Blocos.

Fonte: O autor

No diagrama, é colocada de maneira gráfica a estrutura do sistema e seus relacionamentos. Portanto, a estrutura de autenticação, ao ser validada, direciona para o painel do usuário de acordo com o tipo de usuário autenticado. Para o caso do aluno, ele pode editar resumos e submetê-los para análise, submissão que é enviada para o professor responsável pela disciplina. O professor, por sua vez, pode avaliar os trabalhos submetidos pelos alunos, enquanto o coordenador pode selecionar esses trabalhos que foram aprovados pelo professor responsável e gerar o caderno de resumos.

4.3.2 Diagrama de Caso de uso

Para melhor visualização das ações que cada tipo de usuário pode realizar no sistema, foi desenvolvido o diagrama de caso de uso que modela as ações possíveis no sistema e quem tem permissão de realizá-las (BOOCH; RUMBAUGH; JACOBSON, 2006).

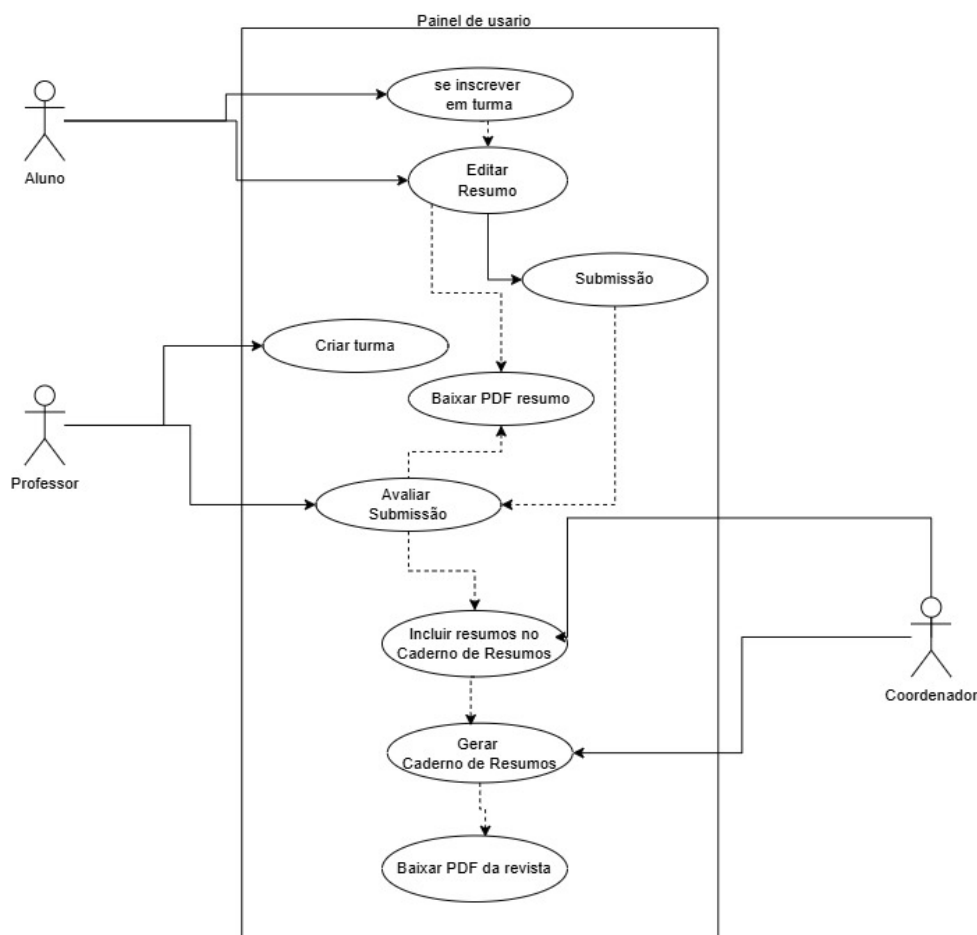


Figura 5 – Diagrama de caso de uso.

Fonte: O autor

4.3.3 Diagrama de Fluxo de dados

O diagrama de Fluxo de dados ou fluxograma, permite visualizar o ciclo de vida de um processo no sistema (BOOCH; RUMBAUGH; JACOBSON, 2006), nesse caso foi aplicado para modelar o processo de edição dos resumos e geração do caderno de resumos, separados por tipo de usuário.

O processo de edição e submissão começa quando o aluno seleciona um de seus trabalhos salvos. As alterações podem ser feitas de forma gradual, permitindo que o usuário salve o progresso antes do envio definitivo. Após a submissão, se o resumo for aprovado, o fluxo de trabalho se encerra; caso seja reprovado, o documento retorna ao status de edição para que o estudante faça os ajustes necessários (Figura 6).

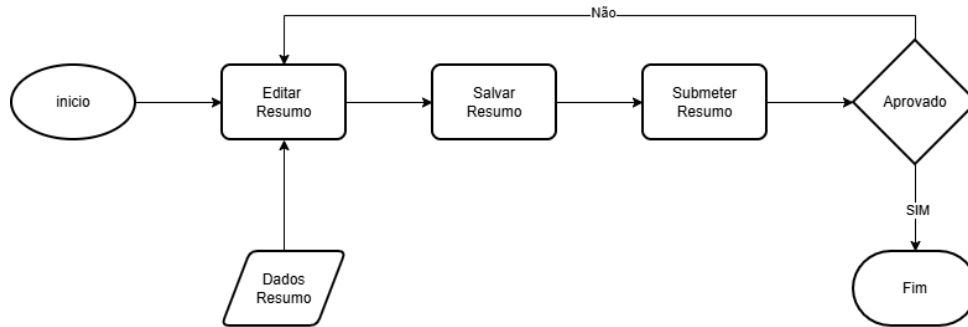


Figura 6 – Fluxograma do resumo para o Aluno.

Fonte: O autor

O processo de avaliação de resumos começa com o professor visualizando os resumos submetidos. Em seguida, ele pode gerar um PDF desse resumo e fornecer um parecer à submissão (Figura 7).

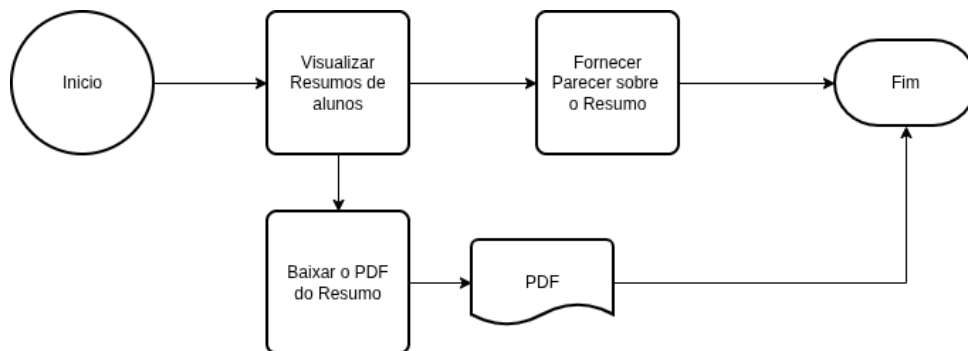


Figura 7 – Fluxograma de avaliação de resumos para o professor.

Fonte: O autor

O processo de geração do caderno de resumos começa com o coordenador visualizando as produções aprovadas. Em seguida, ele seleciona quais trabalhos devem compor o documento e aciona o comando para gerar o arquivo final (Figura 8).

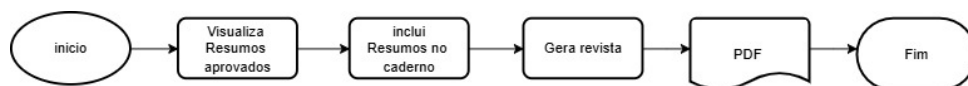


Figura 8 – Fluxograma do Caderno de Resumos para o coordenador.

Fonte: O autor

4.3.4 Diagrama de sequência

O diagrama de sequência é fundamental para mapear a ordem cronológica das interações entre os componentes do sistema. Ele ajuda a visualizar o passo a passo que cada usuário realiza, evidenciando como as mensagens e os dados trafegam pelas classes ou módulos até que o objetivo final daquela funcionalidade seja atingido (BOOCH; RUMBAUGH; JACOBSON, 2006).

O primeiro diagrama desenvolvido foi o do usuário do tipo aluno. Suas permissões incluem realizar o login no sistema, inscrever-se em turmas, editar resumos, fazer o download do PDF de suas produções e, por fim, submeter os trabalhos para a avaliação (Figura 9).

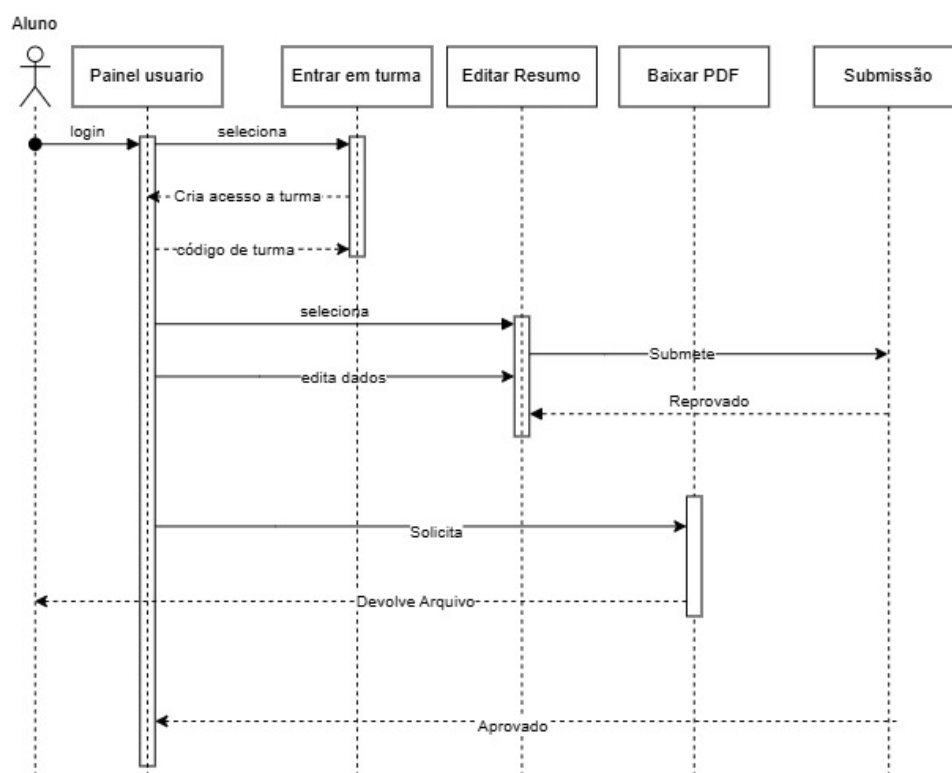


Figura 9 – Diagrama de sequência de aluno.

Fonte: O autor

O segundo diagrama desenvolvido foi o do perfil do professor. Esse usuário tem permissão para criar turmas, visualizar as submissões vinculadas às suas disciplinas, fazer o download dos PDFs e avaliar os resumos enviando um *feedback* aos alunos (Figura 10).

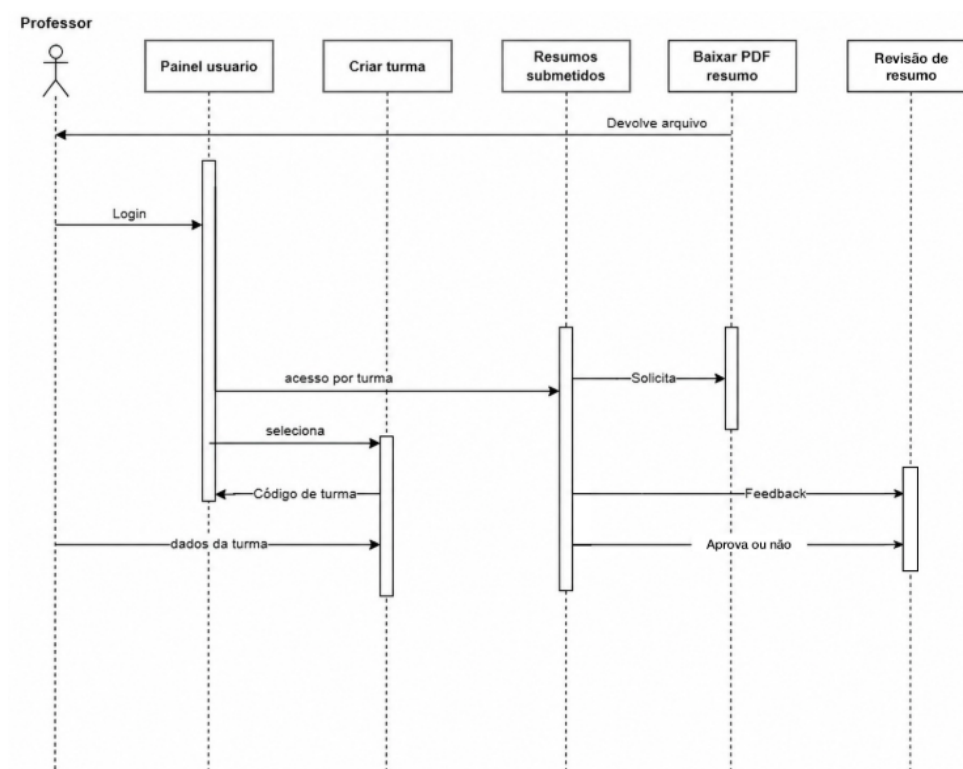


Figura 10 – Diagrama de sequência de Professor.

Fonte: O autor

O último perfil de usuário mapeado é o do coordenador, que possui permissões para visualizar os trabalhos aprovados e incluí-los no caderno de resumos. Além disso, ele pode fazer o download dos arquivos individuais, gerar a coletânea completa e baixar o PDF final do caderno de resumos (Figura 11).

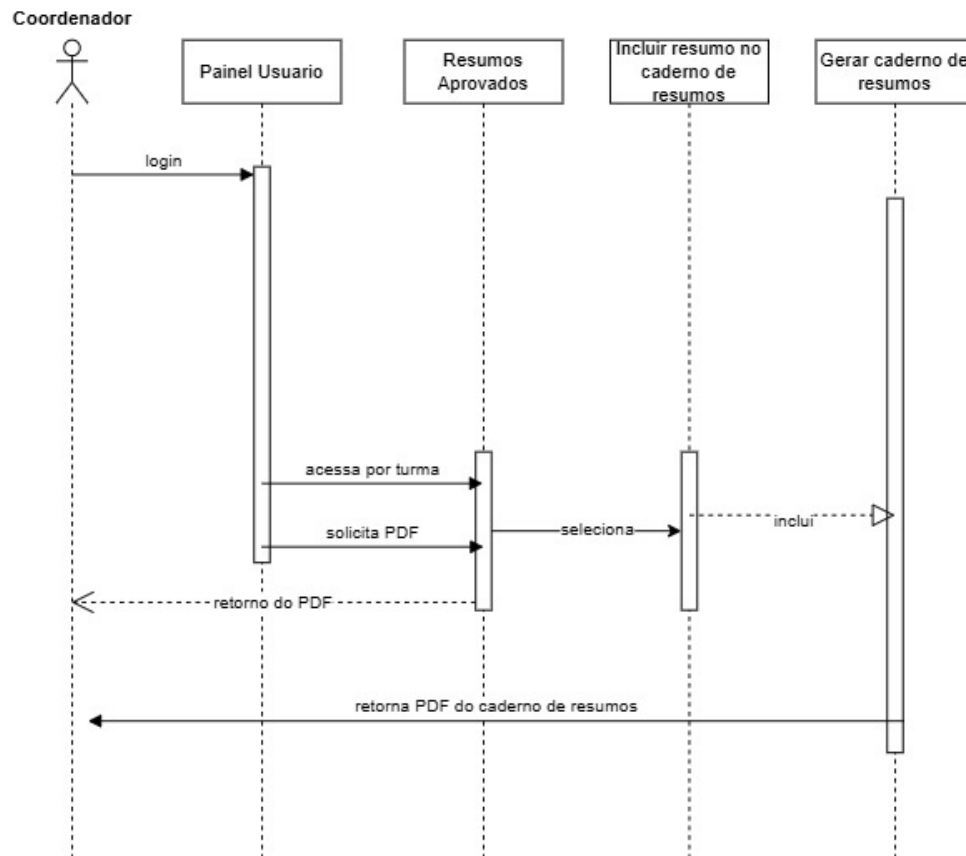


Figura 11 – Diagrama de sequência de Coordendador.

Fonte: O autor

4.4 Criação do projeto em Django e Pipenv

Para o gerenciamento do projeto, optou-se pela utilização do Pipenv. A escolha dessa ferramenta justifica-se pela capacidade de encapsular o ambiente virtual, o que evita a necessidade de reinstalar manualmente as dependências sempre que o desenvolvimento é transferido para uma nova máquina. Além disso, essa prática padroniza o ecossistema do projeto, simplificando significativamente o processo de *deployment* (hospedagem) da aplicação.

No que tange à gestão do código-fonte, foi estruturado um repositório Git (TOKUMOTO, 2026a). A adoção do controle de versão permitiu não apenas rastrear o histórico de modificações do sistema, mas também viabilizou o desenvolvimento em diferentes estações de trabalho, garantindo a integridade e a confiabilidade do código ao longo das etapas do projeto.

Como base para o desenvolvimento da aplicação, optou-se pelo *framework* Django, construído em Python. A escolha fundamenta-se no fato de ser uma tecnologia robusta, consolidada no mercado de desenvolvimento de software e que apresenta uma curva de aprendizado favorável, permitindo uma construção ágil e estruturada do sistema.

4.5 Quadro kanban de controle do desenvolvimento

Para o controle e a priorização das etapas de codificação, foi utilizado o modelo Kanban por meio das funcionalidades de gerenciamento do GitHub. Essa abordagem garantiu um controle dinâmico sobre o ciclo de vida do software, conforme apresentado na Figura 12.

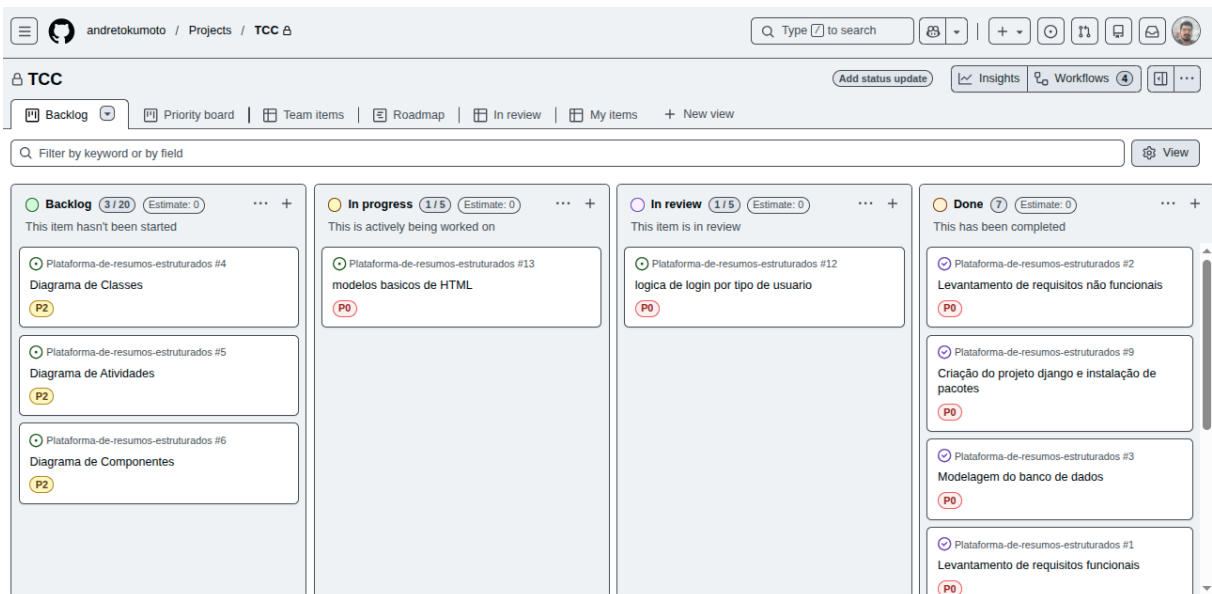


Figura 12 – Controle das etapas com kanban.

Fonte: O autor

4.6 Banco de dados

Concluídas as configurações iniciais, a primeira etapa do desenvolvimento concentrou-se na modelagem do banco de dados. Devido ao suporte nativo do Django ao SQLite, a estrutura de dados foi implementada diretamente por meio do mapeamento objeto-relacional fornecido pelo *framework*, concentrado no arquivo *models.py*. O processo iniciou-se com a importação das bibliotecas e dependências necessárias para a construção das tabelas.

```
models.py x
Plataforma-de-resumos-estruturados > plataformaResumosAPP > models.py > ...
1 from django.db import models
2 from django.contrib.auth.models import AbstractUser
3
```

Figura 13 – Bibliotecas de banco de dados importados

Fonte: O autor

Para a parte de estrutura de usuário foi utilizado a biblioteca *AbstractUser*, que já cria as estruturas de usuário padronizados de maneira a facilitar a parte de autenticação e seção de usuário.

Durante o desenvolvimento foi detectado a necessidade de novas tabelas para armazenar informações padrões de tipos de ODS e uma tabela relacionada para armazenar o programa de cada resumo, para que a escolha de múltiplas opções dessas por alunos ficasse de mais fácil desenvolvimento.

```
class Programa(models.Model):  
    nome = models.CharField(  
        max_length=255,  
        unique=True  
    )  
  
    def __str__(self):  
        return self.nome  
  
class ODS(models.Model):  
    numero = models.IntegerField(  
        unique=True  
    )  
  
    nome = models.CharField(  
        max_length=255  
    )  
  
    def __str__(self):  
        return f'ODS {self.numero} - {self.nome}'
```

Figura 14 – Tabelas criadas durante o desenvolvimento

Fonte: O autor

4.7 Autenticação de usuário

A rotina de autenticação do sistema foi desenvolvida com o suporte da biblioteca *django.contrib.auth*, componente padrão do *framework* que otimiza a segurança da aplicação. Essa ferramenta foi integrada para abstrair a complexidade dos mecanismos de autenticação, sendo responsável desde a geração das interfaces de validação até a verificação de credenciais no banco de dados, além de manter uma seção de usuário.

Como, por padrão, a autenticação de usuário nessa biblioteca ocorre por meio do nome de usuário, foi acrescentada na função de autenticação em *views.py* uma conferência do nome de usuário por e-mail. Além disso, foi acrescentada uma verificação no tipo de

usuário para que o redirecionamento fosse para o painel do tipo de usuário correspondente.

```
user = authenticate(
    request,
    username=username,
    password=password
)

if user:
    login(request, user)

    #Redirecionamento para dashboard por tipo de usuario
    if user.tipo_usuario == 1:
        return redirect('dashboard_aluno')
    elif user.tipo_usuario == 2:
        return redirect('dashboard_professor')
    elif user.tipo_usuario == 3:
        return redirect('dashboard_coordenador')
    else:
        form.add_error(None, "Usuário ou senha incorretos.")
```

Figura 15 – verificação das credenciais e redirecionamento por tipo de usuário

Fonte: O autor

4.8 Criação de turma e matricula em turma por aluno

4.8.1 Criação de turma

Para o desenvolvimento da funcionalidade de criação de turmas, primeiramente, foi desenvolvido um formulário , com a ferramenta de *form* do próprio *framework*, para que fosse realizado a validação dos dados a serem salvos no banco de dados de maneira mais simplificada.

Na sequência, foi elaborada a lógica para a criação de turmas. Se os dados enviados pelo usuário estiverem em conformidade, o sistema associa automaticamente o responsável e o semestre atual com base nos registros do banco de dados. Visando tornar a matrícula mais ágil, incorporou-se também uma rotina que gera um código para a turma, servindo como chave de acesso para que os alunos efetuem sua própria inscrição.

```
while True:
    codigo = ''.join(random.choices(string.ascii_uppercase + string.digits,
                                    k=6))
    if not Turma.objects.filter(codigo_acesso=codigo).exists():
        break
```

Figura 16 – Geração de chave de acesso aleatória para nova turma

Fonte: O autor

Esse código é formado por uma sequência aleatória de dígitos, que conta com a verificação no banco para que seja uma chave única por turma.

4.8.2 Matrícula em turma por aluno

Para a função de matrícula do aluno na turma, foi empregada uma lógica semelhante, com o uso de formulário para validação dos dados. Para localizar a turma, realiza-se uma busca na tabela correspondente do banco de dados por meio da chave de acesso. Caso a chave exista e o aluno ainda não esteja matriculado, a inscrição é realizada. Por outro lado, caso a chave não exista ou o aluno já possua inscrição nessa turma, o sistema recusa a solicitação e retorna uma mensagem de erro para o usuário.

```
try:
    turma = Turma.objects.get(codigo_acesso=codigo)

    Inscricao.objects.create(aluno=request.user, turma=turma)
    messages.success(request, f"Inscrição realizada com sucesso na turma: {turma.nome_turma}")
    return redirect('dashboard_aluno')

except Turma.DoesNotExist:
    messages.error(request, "Código de acesso inválido.")
except IntegrityError:
    messages.error(request, "Você já está inscrito nesta turma.")
```

Figura 17 – Rotina de busca e validação de turma pela chave de acesso
Fonte: O autor

4.9 Formulário de resumo e rotina de submissão para avaliação

Para a criação do formulário de edição de resumos, também foi empregado o uso da ferramenta de formulários, enquanto as informações de aluno e turma do projeto foram adicionadas através de um relacionamento com as respectivas tabelas correspondentes.

Justamente na implementação desse formulário que a ideia de tabelas para ODS e programa do projeto surgiu, para facilitar a implementação e deixar o código mais limpo.

Foi criada uma validação nessa função, para que, caso o resumo esteja com *status* de 'em avaliação' ou 'aprovado', o aluno não possa realizar edições, a fim de manter a integridade da avaliação pelo professor.

```
if projeto.status_projeto not in [0, 3]:
    messages.error(request, "Este relatório não pode ser editado pois já foi submetido.")
    return redirect('relatorio_projeto', inscricao_id=inscricao.id)
```

Figura 18 – Validação do status de avaliação do resumo
Fonte: O autor

Para a rotina de submissão de resumo, a função verifica o seu *status*. Se a situação for 'rascunho' ou 'rejeitado', a função cria um novo registro de submissão para o documento, além de modificar o campo de *status* na tabela do resumo para 'em análise'.

4.10 Avaliação de submissão e feedback

Para desenvolver a funcionalidade de avaliação de projetos pelo professor, primeiramente, foi desenvolvida a função para retornar ao painel do professor apenas as turmas criadas por ele. A rotina localiza as turmas cujo responsável seja o usuário conectado e, para cada uma delas, busca os projetos com submissão, retornando as listas com as informações encontradas.

Na função de avaliação, são retornadas para o professor as informações do resumo selecionado, apenas para visualização. O professor pode atribuir duas avaliações à submissão: 'aprovado' ou 'rejeitado'; para ambas, é possível registrar um parecer (*feedback*) que fica salvo na tabela de submissão e pode ser visualizado pelo aluno.

4.11 Visualização de resumos aprovados e inclusão na revista no painel de coordenador

Para a exibição dos resumos aprovados, o sistema executa uma consulta na base de dados, filtrando os projetos que apresentam o *status* 'aprovado' e retornando uma listagem com os dados pertinentes. A partir dessa interface, o coordenador possui a prerrogativa de ler os relatórios e, se julgar pertinente, incluí-los na revista do semestre.

Essa inserção ocorre por meio da criação de um registro desse resumo na tabela de projetos selecionados. Caso a tabela de revistas esteja vazia, o sistema cria uma nova entrada para inicializar a edição da revista, incluindo em seguida o dado na tabela de resumos selecionados que está vinculada ao semestre vigente.

4.12 Geração do PDF

Para a funcionalidade de geração de PDF foi escolhido o uso do LaTeX, por entregar uma alta qualidade visual, além de facilitar na questão de formatação do produto. Para isso foi empregado o compilador Python *pdflatex*.

4.12.1 Manipulação do arquivo *.tex* para resumo

O primeiro passo para o desenvolvimento dessa etapa foi a criação de um arquivo *.tex* base. Para isso, estruturou-se o documento com valores padrão definidos para cada

seção constitutiva do resumo.

```
\noindent
\textbf{Disciplina e Curso}

_DISCIPLINA_

\vspace{0.7cm}
```

Figura 19 – Seção do resumo com valores padrão

Fonte: O autor

Para a geração de um arquivo *.tex* com base no *template*, foi implementada uma função que recebe os parâmetros provenientes do módulo *views*. De acordo com a categoria do documento ,seja ele resumo ou revista, o sistema substitui os valores padrão do modelo e salva o novo arquivo *.tex* em um diretório temporário.

```
if document_type == 'resumo':
    template_path = os.path.join(BASE_DIR, 'arquivosTEX', 'resumo.tex')

    doc_gerado = _open_template(template_path)
    doc_gerado = doc_gerado.replace('_TITULO_RESUMO_', dados.get('titulo_resumo', ''))
    doc_gerado = doc_gerado.replace('_AUTORES_', dados.get('autores', ''))
    doc_gerado = doc_gerado.replace('_DISCIPLINA_', dados.get('disciplina', ''))
    doc_gerado = doc_gerado.replace('_PROGRAMA_PEPIC_', dados.get('pepict', ''))
    doc_gerado = doc_gerado.replace('_OBJETIVOS_', dados.get('objetivos', ''))
    doc_gerado = doc_gerado.replace('_METODOLOGIA_', dados.get('metodologia', ''))
    doc_gerado = doc_gerado.replace('_RESULTADOS_', dados.get('resultados', ''))
    doc_gerado = doc_gerado.replace('_ODS_', dados.get('ods', ''))
    doc_gerado = doc_gerado.replace('_REFLEXAO_', dados.get('reflexao', ''))
    doc_gerado = doc_gerado.replace('_REFERENCIAS_', dados.get('referencias', ''))

    _salva_arquivo(output_path, doc_gerado)
```

Figura 20 – Função de criação do *.tex* de resumo

Fonte: O autor

4.12.2 Manipulação do arquivo *.tex* para revista

A lógica básica para a geração do arquivo *.tex* da revista é a mesma. Porém, como a revista recebe dados de diferentes tabelas, foi criada uma rotina para agregar essas informações e estruturar o documento final. A solução mais interessante encontrada foi através da manipulação de arquivos de texto.

Para viabilizar esse processo, foi desenvolvida uma função que percorre todos os projetos registrados na tabela de resumos incluídos na revista. Simultaneamente, o arquivo de texto é estruturado através do seguinte processo de execução:

1. **Criação de Capítulos por Turma:** Para cada turma identificada no banco de dados, o algoritmo insere no arquivo o comando em LaTeX necessário para a criação de um novo capítulo, utilizando o nome da própria turma como título.
2. **Estruturação de Seções por Resumo:** Nas linhas subsequentes, o sistema injeta as instruções em LaTeX para a abertura de uma nova seção, cujo título corresponde ao nome do resumo.
3. **Inclusão dos Dados do Resumo:** Nas linhas imediatamente abaixo da seção criada, os dados constitutivos do resumo são inseridos de forma sequencial e ordenada.

```

Plataforma-de-resumos-estruturados > plataformaResumosAPP > arquivos_temp > ≡ conteudo_revista.txt
1  \chapter{Arquitetura e Organização de Computadores}
2
3  \section{Projeto - Panda}
4  \noindent
5  \textbf{Autores}
6  Ana Carolina
7
8  \vspace{0.7cm}
9  \noindent
10 \textbf{Disciplina e Curso}
11 Engenharia de Computação
12
13 \vspace{0.7cm}
14 \noindent
15 \textbf{Programa PEP ICT vinculado}
16 Tecnologia e Inovação para o Desenvolvimento Social
17
18 \vspace{0.7cm}
19 \noindent
20 \textbf{Objetivos}
21 muitos
22
23 \vspace{0.7cm}
24 \noindent
25 \textbf{Metodologia}
26 o que eu quiser

```

Figura 21 – Trecho do arquivo texto criado na compilação do caderno de resumos

Fonte: O autor

Após esse processo, a função de manipulação do arquivo *.tex* é executada. Essa rotina acessa o diretório, faz a leitura do arquivo de texto e armazena os dados em uma variável. O procedimento de manipulação segue o mesmo padrão do resumo: os dados padrão no *template* são substituídos pelos dados recebidos e pelos dados lidos do arquivo.

4.13 Compilação do PDF

A função que realiza a compilação do PDF recebe os dados vindos do módulo *views* (informações estas lidas do banco de dados) e faz a chamada da função de geração do arquivo *.tex*, já descrita anteriormente. Após receber o retorno, a função define o caminho

para o novo arquivo a ser gerado no mesmo local do arquivo *.tex*. Em seguida, utiliza-se a ferramenta *pdflatex* para gerar o PDF a partir do arquivo *.tex* obtido.

```
try:
    result = subprocess.run(
        ['pdflatex', '-interaction=nonstopmode', '-output-directory', output_dir,
         tex_path],
        check=True,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
```

Figura 22 – Compilação do PDF

Fonte: O autor

Importante ressaltar que tanto a rotina de geração do PDF do resumo quanto a da revista utilizam o mesmo método de compilação de PDF. A distinção ocorre apenas pelo envio de dados diferentes, acompanhados da descrição do tipo de arquivo correspondente.

```
dados = {
    'document_type': 'revista',
    'titulo_revista': revista.titulo,
    'data': data_formatada,
}

try:
    pdf_gerado = to_pdf(dados)

    if os.path.exists(pdf_gerado):
        response = FileResponse(open(pdf_gerado, 'rb'), as_attachment=True)
        response['Content-Disposition'] = f'attachment; filename="resumo_{revista.id}.pdf"'

        return response

    messages.error(request, "O arquivo PDF não foi encontrado.")

except Exception as e:
    messages.error(request, f"Erro ao gerar o PDF: {str(e)}")
```

Figura 23 – Chamada de função e retorno do arquivo gerado

Fonte: O autor

4.14 Deploy

Para disponibilizar o sistema na internet, optou-se pela utilização da plataforma de computação em nuvem Fly.io como ambiente de produção. A escolha fundamentou-se no modelo de implantação baseado em containers isolados e na oferta de uma camada gratuita compatível com os requisitos de prototipagem do projeto. Foi escolhido a hospedagem via container (docker) por conta da grande quantidade de recursos necessários.

A configuração do ambiente de hospedagem foi realizada por meio do arquivo *fly.toml*, que centraliza as definições de funcionamento da aplicação. Esse arquivo estabelece os parâmetros necessários para a plataforma rodar corretamente na nuvem, incluindo a alocação de memória e o direcionamento de acessos dos usuários. Para garantir a

estabilidade do sistema, a porta interna do servidor foi configurada para receber requisições na porta 8080, além da inclusão de rotinas automáticas de monitoramento para verificar continuamente se o serviço Django está ativo e funcionando sem erros. Para colocar o sistema na nuvem, foram criados dois arquivos: o *Dockerfile* e o *entrypoint.sh*. O *Dockerfile* prepara o servidor, instalando o Python, o Django e os pacotes do LaTeX para garantir que a geração de PDFs funcione na internet. Já o script *entrypoint.sh* cuida do banco de dados SQLite toda vez que o sistema é ligado, verificando se os dados já existem ou se precisa iniciar um banco novo, evitando a perda de informações.

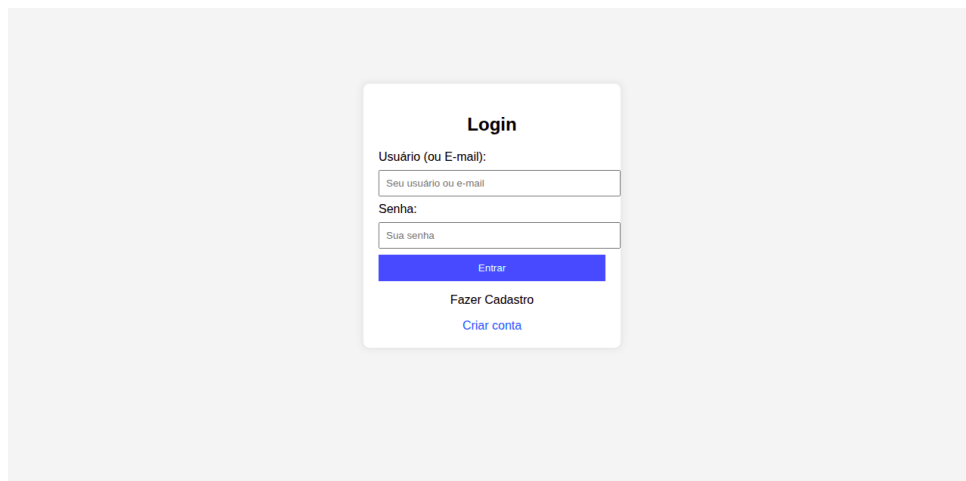
A etapa de hospedagem é fundamental porque transfere o sistema do computador de desenvolvimento para um servidor na internet, tornando-o acessível para o público. No contexto deste trabalho, o deploy permitiu que a plataforma saísse de um ambiente de testes local e ficasse disponível na nuvem.

5 Resultados e Discussão

5.1 Resultados obtidos

5.1.1 *Login* no sistema

Cabe destacar que, por limitações de tempo, a tela de *login* foi a única que recebeu uma camada de estilo. Ela possui dois campos de texto para entrada de credenciais, instruções breves sobre o preenchimento de cada campo, um botão de login e um botão de redirecionamento para a tela de cadastro de usuário.



Login

Usuário (ou E-mail):

Senha:

Entrar

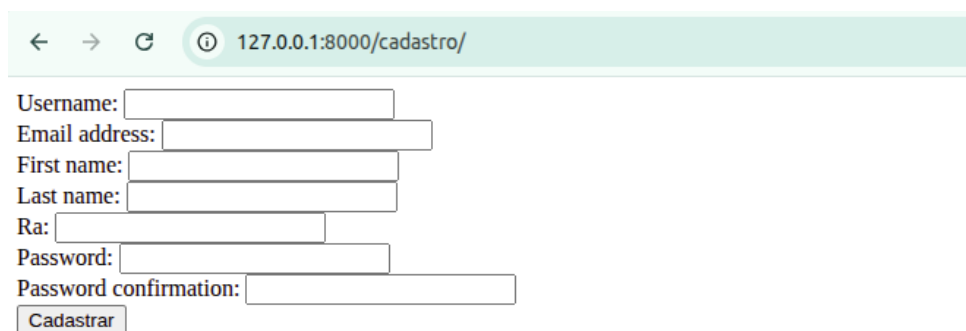
Fazer Cadastro

[Criar conta](#)

Figura 24 – tela de *login*
Fonte: O autor

5.1.2 Cadastro de usuário

A tela de cadastro possui campos de texto identificados, nos quais são solicitados apenas os dados essenciais para este protótipo. .



← → ↻ ⓘ 127.0.0.1:8000/cadastro/

Username:

Email address:

First name:

Last name:

Ra:

Password:

Password confirmation:

Cadastrar

Figura 25 – tela de cadastro de usuário
Fonte: O autor

5.1.3 Perfil de Aluno

5.1.3.1 Painel de aluno

A tela de perfil identifica o painel do aluno e exibe o nome do usuário logado. Na parte inferior, constam a lista de turmas em que ele está inscrito, o *status* de cada resumo e um *link* de acesso. Além disso, a interface disponibiliza botões para inscrição em novas turmas e para encerramento da sessão.



Figura 26 – Painel de Aluno
Fonte: O autor

5.1.3.2 Inscrição em turma

A tela de inscrição apresenta um campo de texto para a inserção do código da turma, além de botões para efetuar a inscrição e para retornar à tela anterior.

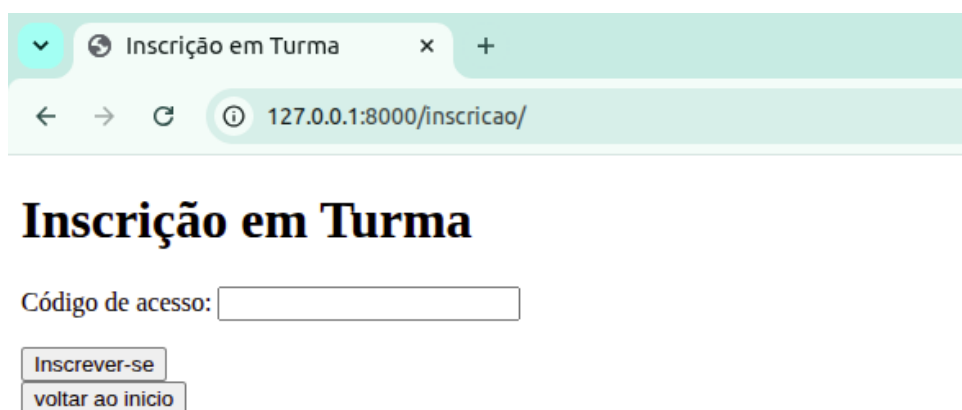


Figura 27 – Tela de inscrição em turma
Fonte: O autor

5.1.3.3 Edição de Resumo Estruturado

Na tela de edição de resumos, é possível visualizar o *status* do documento. Abaixo, constam o campo de texto para a inserção do título e as caixas de seleção para os dados que permitem múltiplas escolhas.

Resumos estruturado

← → ↻ 127.0.0.1:8000/relatorio/21/ ☆ 📄 🌱

Edição de Resumos Estruturados

Status Atual: **Rascunho**

Inscrição realizada com sucesso na turma: projeto 3

Título:

Programas (Selecione):

☒ Educação e Sustentabilidade
☐ Tecnologia e Inovação para o Desenvolvimento Social
Selecione o programa PEP ICT vinculado (Educação e Sustentabilidade ou Tecnologia e Inovação para o Desenvolvimento Social).

ODS (Selecione de 1 a 3):

- ☐ ODS 1 - Erradicação da Pobreza
- ☐ ODS 2 - Fome Zero e Agricultura Sustentável
- ☐ ODS 3 - Saúde e Bem-Estar
- ☐ ODS 4 - Educação de Qualidade
- ☐ ODS 5 - Igualdade de Gênero
- ☐ ODS 6 - Água Potável e Saneamento
- ☐ ODS 7 - Energia Limpa e Acessível
- ☐ ODS 8 - Trabalho Decente e Crescimento Econômico
- ☐ ODS 9 - Indústria, Inovação e Infraestrutura
- ☐ ODS 10 - Redução das Desigualdades
- ☐ ODS 11 - Cidades e Comunidades Sustentáveis
- ☐ ODS 12 - Consumo e Produção Responsáveis
- ☐ ODS 13 - Ação Contra a Mudança Global do Clima
- ☐ ODS 14 - Vida na Água

Figura 28 – Tela de edição de Resumo - parte 1

Fonte: O autor

Os campos destinados a textos livres contêm instruções sobre o conteúdo esperado em cada espaço. Abaixo, estão disponíveis os botões para salvar e continuar a edição posteriormente, submeter o resumo para análise e baixar o arquivo em formato PDF.

Resultados:

✎ Liste os produtos gerados (protótipos, oficinas, relatórios, etc.) e os resultados alcançados.

Justificativa ODS:

✎ Indique até 3 ODS trabalhados e justifique como o trabalho contribuiu com cada um deles.

Reflexão:

✎ Relate os principais aprendizados, desafios e percepções sobre a extensão universitária.

Referências:

✎ Inclua as fontes bibliográficas utilizadas (se houver, no formato ABNT).

[Voltar](#)

Figura 29 – Tela de edição de Resumo - parte 2

Fonte: O autor

Caso o resumo esteja em análise, o aluno não poderá editá-lo. As informações serão exibidas em cinza-claro e ficarão bloqueadas para modificação, permitindo apenas a

visualização e o download do PDF.

Edição de Resumos Estruturados

Status Atual: **Em análise**

Projeto submetido com sucesso!

Título:

Programas (Selecione):
☒ Educação e Sustentabilidade
☐ Tecnologia e Inovação para o Desenvolvimento Social
Selecione o programa PEP ICT vinculado (Educação e Sustentabilidade ou Tecnologia e Inovação para o Desenvolvimento Social).

ODS (Selecione de 1 a 3):
☐ ODS 1 - Erradicação da Pobreza
☐ ODS 2 - Fome Zero e Agricultura Sustentável
☐ ODS 3 - Saúde e Bem-Estar
☐ ODS 4 - Educação de Qualidade
☐ ODS 5 - Igualdade de Gênero
☐ ODS 6 - Água Potável e Saneamento
☐ ODS 7 - Energia Limpa e Acessível
☒ ODS 8 - Trabalho Decente e Crescimento Econômico
☐ ODS 9 - Indústria, Inovação e Infraestrutura
☒ ODS 10 - Redução das Desigualdades
☐ ODS 11 - Cidades e Comunidades Sustentáveis
☐ ODS 12 - Consumo e Produção Responsáveis
☐ ODS 13 - Ação Contra a Mudança Global do Clima
☐ ODS 14 - Vida na Água

Figura 30 – Tela de edição de Resumo com trabalho submetido

Fonte: O autor

5.1.4 Perfil de Professor

5.1.4.1 Painel de Professor

A tela do painel do professor exibe uma lista com as turmas criadas por ele e seus respectivos códigos. No canto inferior, há um botão que redireciona o usuário para a tela de criação de turmas.

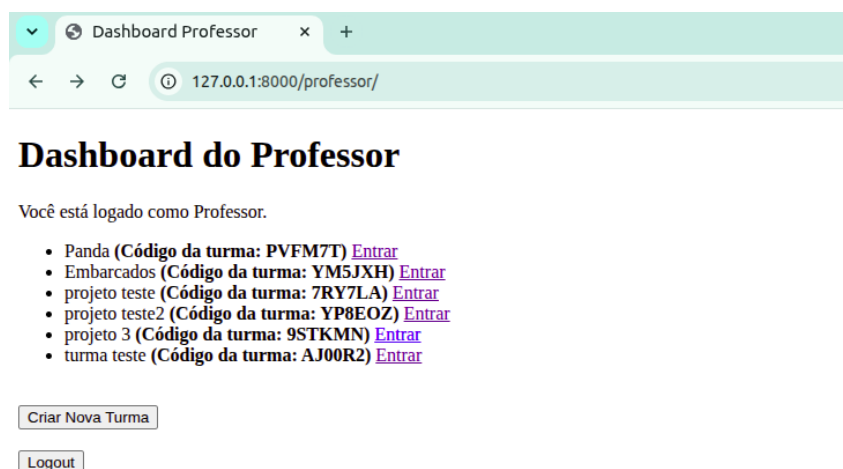
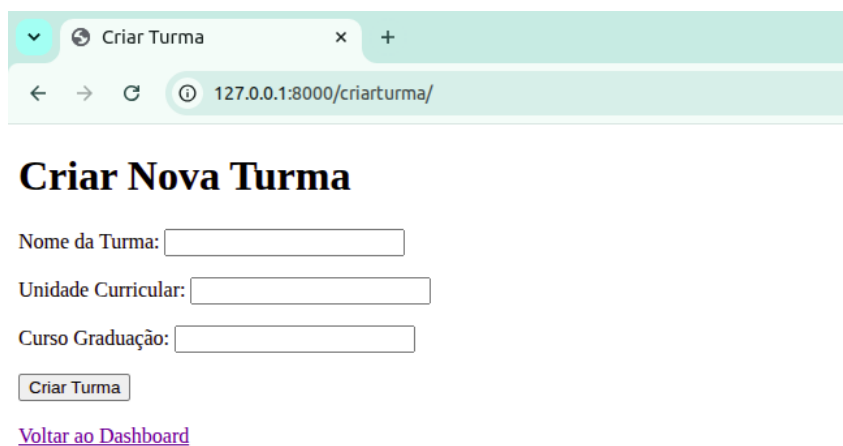


Figura 31 – Painel de Professor

Fonte: O autor

5.1.4.2 Criação de Turma

A tela de criação de turmas apresenta campos de texto para o preenchimento das informações necessárias. Na parte inferior, há botões para retornar à tela anterior e para confirmar a criação da turma.



Criar Nova Turma

Nome da Turma:

Unidade Curricular:

Curso Graduação:

[Voltar ao Dashboard](#)

Figura 32 – Tela de criação de nova turma
Fonte: O autor

5.1.4.3 Painel da turma

Ao acessar a turma, o professor visualiza as informações dos resumos submetidos para aquela sala. A interface exibe uma lista de trabalhos com dados sobre o autor, o título e o status de cada resumo, permitindo também acessar os envios individualmente.



Embarcados

Disciplina: Sistemas Embarcados
Curso: Engenharia de Computação

Resumos Submetidos

Aluno	Título do Projeto	Status	Ação
Aluno Teste2	Titulo do projeto	Aprovado	Ver / Baixar PDF
Ana Carolina	reenvio	Aprovado	Ver / Baixar PDF
Tom Felinos	Novo Projeto	Aprovado	Ver / Baixar PDF

[Voltar ao Dashboard](#)

Figura 33 – Tela de painel da turma
Fonte: O autor

5.1.4.4 Tela de análise do resumo

Ao selecionar um resumo, o professor pode visualizar todas as informações da submissão. No entanto, o sistema não permite a edição do documento por parte do docente em nenhum momento.

Projeto - projeto 3

Aluno: Aluno Teste2

Autores: Aluno Teste2

Turma: projeto 3

Programas: Educação e Sustentabilidade

ODS: ODS 8 - Trabalho Decente e Crescimento Econômico, ODS 10 - Redução das Desigualdades

Objetivos

teste

Metodologia

teste

Resultados

teste

Justificativa ODS

teste

Figura 34 – Tela de análise do resumo - parte 1

Fonte: O autor

Na parte inferior da tela, há um campo destinado ao *feedback* do professor para o aluno, além de botões para o envio da avaliação.

Reflexão

teste

Referências

teste

[Baixar PDF do Resumo](#)

Avaliação

Feedback:

[Aprovar](#) [Rejeitar](#)

[Voltar ao painel](#)

Figura 35 – Tela de análise do resumo - parte 2

Fonte: O autor

5.1.5 Perfil de coordenador

5.1.5.1 Painel de Coordenador

O painel do coordenador disponibiliza opções para acessar as turmas vigentes, entrar na tela de gerenciamento de usuários e gerar a revista.

Dashboard do Coordenador

Você está logado como Coordenador.

[Ver Turmas e Projetos Aprovados](#) [Gerenciar tipos de usuários](#)

Caderno de Resumos

Caderno de Resumos	Semestre	Projetos	Ação
Caderno 2026/1	2026/1	8	Gerar PDF do Caderno de resumos

[Logout](#)

Figura 36 – Tela de painel do Coordenador
Fonte: O autor

5.1.5.2 Painel de Turmas Vigentes

Na tela de visualização de turmas, o coordenador pode listar as turmas ativas e acessar a relação de resumos de cada uma delas individualmente, além de poder gerar o PDF do caderno de resumos.

[← Voltar ao Dashboard](#)

Gestão de Semestres e Turmas

Veja abaixo a divisão das turmas e a respectiva gerência de Caderno de resumos configuradas para cada semestre.

2026/1º Semestre

Caderno de Resumos Associado: Revista 2026/1

Status: 8 projeto(s) associado(s).

[Gerar PDF desta Revista](#)

Turmas Cadastradas neste Semestre:

- Panda — Arquitetura e Organização de Computadores (Engenharia de Computação) [Ver Resumos Aprovados](#)
- Embarcados — Sistemas Embarcados (Engenharia de Computação) [Ver Resumos Aprovados](#)
- projeto teste — UC teste (BCT) [Ver Resumos Aprovados](#)
- projeto teste2 — UC teste 2 (BCT) [Ver Resumos Aprovados](#)
- projeto 3 — Banco de dados (BCT) [Ver Resumos Aprovados](#)
- turma teste — UC teste 2 (BCT) [Ver Resumos Aprovados](#)

Figura 37 – Tela de painel de turma vigentes
Fonte: O autor

5.1.5.3 Painel de Resumos aprovados da turma

Ao selecionar uma turma o coordenador vai ter uma lista com os resumos aprovados daquela turma, com um link para acessar o conteúdo do resumo e informações básicas de cada resumo.

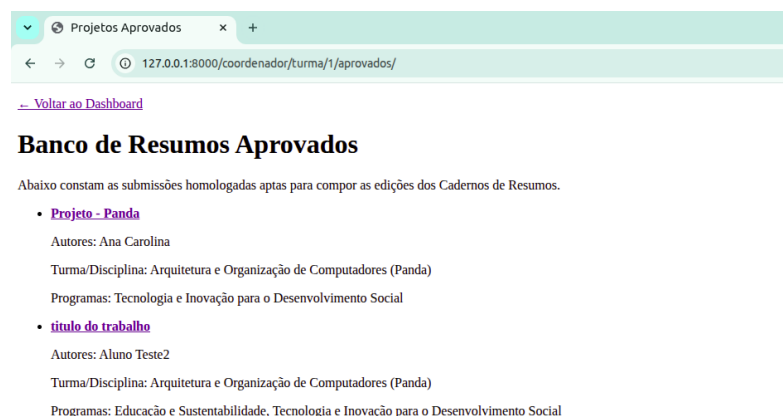


Figura 38 – Tela de painel de resumos aprovados da turma
Fonte: O autor

5.1.5.4 Gerenciar tipo de usuário

O coordenador tem acesso a uma tela na qual pode pesquisar qualquer usuário por seu nome de registro e alterar o tipo de perfil entre 'professor' e 'aluno'.



Figura 39 – Tela de painel de gerenciamento de tipo de usuário
Fonte: O autor

5.1.6 PDF de Resumo gerado

Os três tipos de usuário possuem acesso à opção de baixar o PDF do resumo. Esse documento é compilado por meio do *pdfLaTeX*, gerando como resultado o seguinte arquivo (Figura ??) :

UNIVERSIDADE FEDERAL DE SÃO PAULO – UNIFESP

PROGRAMA PEP ICT

ICT – Instituto de Ciência e Tecnologia

titulo do trabalho**Autores(as)**

Aluno Teste2

Disciplina e Curso

Arquitetura e Organização de Computadores

Programa PEP ICT vinculado

Educação e Sustentabilidade, Tecnologia e Inovação para o Desenvolvimento Social

Objetivos do Trabalho

sistema deve permitir que o coordenador filtre turmas por semestre, visualize projetos aprovados, selecione os trabalhos que fazem parte da revista e gere o arquivo PDF consolidado da edição.

Metodologia

Concluída a etapa de requisitos, o passo seguinte foi estruturar o modelo de resumo. Para isso, realizamos reuniões com a orientação do projeto para alinhar e selecionar quais informações seriam essenciais e pertinentes para compor esses resumos estruturados, o que resultou no modelo ilustrado na imagem abaixo:

Resultados e Produtos

Foi desenvolvida em 2001, quando seus autores buscaram solucionar lacunas e ineficiências do desenvolvimento em cascata, detectadas quando essa metodologia não produzia mais resultados satisfatórios frente a vários parâmetros de medida. Era comum, por exemplo, demorar anos entre a identificação de uma necessidade da empresa e a entrega do sistema pronto. Nesse meio-tempo, as mudanças no mercado e nas exigências do negócio

1

Figura 40 – PDF do resumo gerado pelo sistema

Fonte: O autor

5.1.7 PDF do caderno de resumos

O coordenador pode gerar o caderno de resumos com os arquivos selecionados previamente em seu perfil. Como resultado, o sistema compila e disponibiliza o PDF do Caderno de resumos. A estrutura inicial do arquivo gerado compreende a capa, a folha de

rosto e os dados informativos da publicação, configurando a identidade visual padrão do Caderno de Resumos.

UNIVERSIDADE FEDERAL DE SÃO PAULO - UNIFESP

PROGRAMA PEP ICT

Caderno 2026/1

Caderno de Resumos Estruturados dos projetos PEP ICT

São José dos Campos

2026

Figura 41 – Capa do Caderno de Resumos
Fonte: O autor

UNIVERSIDADE FEDERAL DE SÃO PAULO - UNIFESP

PROGRAMA PEP ICT

Caderno 2026/1

Revista institucional destinada à publicação
de relatórios acadêmicos desenvolvidos por
alunos vinculados às unidades curriculares
e projetos do programa PEP ICT.

São José dos Campos
2026

Figura 42 – Contracapa do Caderno de Resumos
Fonte: O autor

Informações da publicação

UNIVERSIDADE FEDERAL DE SÃO PAULO – UNIFESP

Programa PEP ICT

NOME_COORDENADOR_GERAL_PEP ICT

Coordenação Editorial

NOME_COORDENADOR_EDITORIAL

Comissão Organizadora

NOME_COMISSAO

Revisão

NOME_REVISAO

Diagramação

NOME_DIAGRAMACAO

Revista 2026/1

2026

ORGANIZADORES

Figura 43 – Contracapa do Caderno de Resumos
Fonte: O autor

O sumário foi um ponto de dificuldade, a compilação dessa parte estava bloqueando a geração do PDF do caderno de resumos, por isso a compilação acontece sem o pacote ABNT o que ocasiona a não geração do sumário.

O capítulo inicial compreende a introdução do Caderno, permitindo a inserção de textos livres por parte da coordenação. Cabe destacar que este recurso específico ainda não se encontra implementado na versão atual do protótipo.

Capítulo 1

Introdução

INTRODUCAO_R EVISTA

Figura 44 – Introdução do Caderno de Resumos
Fonte: O autor

Nas páginas seguintes, os resumos selecionados são apresentados de forma organizada por turma, mantendo a mesma estrutura básica do modelo individual.

2.3 título do trabalho

Autores Aluno Teste2

Disciplina e Curso Engenharia de Computação

Programa PEP ICT vinculado Educação e Sustentabilidade, Tecnologia e Inovação para o Desenvolvimento Social

Objetivos sistema deve permitir que o coordenador filtre turmas por semestre, visualize projetos aprovados, selecione os trabalhos que farão parte da revista e gere o arquivo PDF consolidado da edição.

Metodologia Concluída a etapa de requisitos, o passo seguinte foi estruturar o modelo de resumo. Para isso, realizamos reuniões com a orientação do projeto para alinhar e selecionar quais informações seriam essenciais e pertinentes para compor esses resumos estruturados, o que resultou no modelo ilustrado na imagem abaixo:

Resultados e Produtos Foi desenvolvida em 2001, quando seus autores buscaram solucionar lacunas e ineficiências do desenvolvimento em cascata, detectadas quando essa metodologia não produzia mais resultados satisfatórios frente a vários parâmetros de medida. Era comum, por exemplo, demorar anos entre a identificação de uma necessidade da empresa e a entrega do sistema pronto. Nesse meio-tempo, as mudanças no mercado e nas exigências do negócio faziam com que grande parte do projeto fosse cancelada antes mesmo de ser entregue. Esse desperdício de tempo e dinheiro motivou vários desenvolvedores a buscarem uma alternativa.

Relação com os ODS A metodologia ágil é uma abordagem de desenvolvimento que visa criar um fluxo de trabalho para tornar o processo de produção o mais eficiente e assertivo possível. Nessa abordagem, o desenvolvimento é um processo colaborativo no qual todas as partes mantêm um diálogo claro, a fim de manter o produto o mais próximo do desejado pelo solicitante, dentro da realidade de desenvolvimento e recursos disponíveis

Reflexão Crítica A metodologia ágil é uma abordagem de desenvolvimento que visa criar um fluxo de trabalho para tornar o processo de produção o mais eficiente e assertivo possível. Nessa abordagem, o desenvolvimento é um processo colaborativo no qual todas as partes mantêm um diálogo claro, a fim de manter o produto o mais pró-

Figura 45 – Resumos componentes da publicação

Fonte: O autor

5.2 Sistema hospedado

Após as etapas de configuração de hospedagem do sistema, temos o sistema rodando no servidor *fly.io* (TOKUMOTO, 2026b).

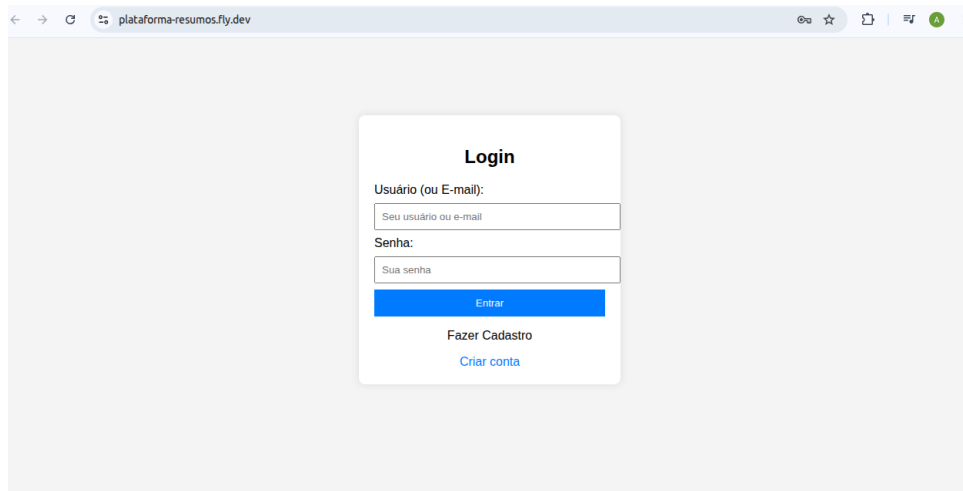


Figura 46 – Sistema Hospedado
Fonte: O autor

5.3 Discussão

A comparação de sistemas correlatos mostrou que as tecnologias existentes, como o *OJS*, o *Janeway*, o *EasyChair* e o *JEMS*, não atendem plenamente às necessidades particulares do campus ICT-UNIFESP. Enquanto essas ferramentas são focadas em eventos pontuais ou periódicos tradicionais, a plataforma desenvolvida preenche essa lacuna de maneira customizada ao integrar-se ao dia a dia das disciplinas semestrais. O grande diferencial técnico do sistema reside na automação do processo de ponta a ponta: o aluno realiza a auto-inscrição por código e preenche um formulário estruturado, que é convertido de forma direta em um arquivo PDF diagramado via LaTeX, além da geração do caderno de resumos pelo coordenador. Dessa forma, o protótipo supera as limitações das plataformas existentes, eliminando a carga de trabalho manual e o custo de ferramentas pagas.

A escolha pelo *Django* se mostrou uma decisão acertada, visto que o framework oferece todos os recursos necessários para o escopo deste projeto, além de apresentar uma estrutura intuitiva e de fácil compreensão, o que otimizou significativamente o processo de desenvolvimento. O SQLite também se revelou uma escolha bem-sucedida, fornecendo o suporte necessário para a elaboração deste protótipo e para a validação de suas funcionalidades. Vale ressaltar, contudo, que a literatura não recomenda seu uso para cenários com alto volume de acessos simultâneos ([SQLite Consortium, 2026](#)). Portanto, para uma futura distribuição em ambiente de produção, será fundamental migrar o sistema para um sistema de gerenciamento de banco de dados que suporte múltiplos acessos, como o *MySQL* ou o *PostgreSQL*.

Paralelamente à escolha das tecnologias, a adoção da metodologia Kanban facilitou de maneira considerável a organização do fluxo de trabalho. Frente ao grande volume de requisitos técnicos, o método visual permitiu uma compreensão clara do estágio atual de

desenvolvimento, tornando evidente quais tarefas já haviam sido concluídas e quais ainda demandavam execução.

A escolha pela lógica de inscrição em turma via código de acesso dá liberdade ao aluno para se inscrever nos projetos de maneira ágil e direta, reduzindo também o trabalho do professor de ter que identificar os alunos dos seus projetos e inscrevê-los um a um, o que, além de trabalhoso, seria mais suscetível a erros do que a maneira descentralizada adotada.

A escolha pela plataforma Fly.io ocorreu porque ela foi a alternativa que melhor ofereceu suporte para volumes persistentes de banco de dados a um custo mais baixo, o que garantiu que as informações do SQLite não fossem perdidas a cada reinicialização. Além disso, o serviço se destacou por ser de fácil configuração, agilizando o processo de colocação do sistema na internet e permitindo o foco no desenvolvimento das funcionalidades principais.

Algumas ideias foram testadas para o desenvolvimento do processo de geração de PDF. A primeira opção consistiu na combinação da biblioteca PyLatex com o PDFLaTeX. Essa biblioteca auxilia na criação e manipulação de arquivos *.tex* com Python; porém, esse processo estava demandando muito tempo de desenvolvimento. Por isso, foi utilizada outra abordagem, adotando arquivos *.tex* padrão e uma função que usasse esse modelo para substituir os valores vindos do banco de dados referentes ao resumo ou caderno de resumos. Essa abordagem trouxe o retorno esperado para a geração do PDF do resumo individual; entretanto, para o caderno de resumos completo, a estratégia direta não funcionou como desejado. Diante disso, adotou-se uma estratégia baseada em um arquivo de texto intermediário para a geração dos capítulos do caderno. Embora não seja a abordagem de melhor desempenho, foi a melhor solução encontrada mediante o tempo limitado de entrega. Futuramente, essa lógica poderá ser revista em busca de uma solução mais eficiente.

Ainda faltam algumas funcionalidades que seriam interessantes, mas que não foram implementadas devido à limitação de tempo. Entre elas, destaca-se a possibilidade de edição de resumos por múltiplos autores. Essa funcionalidade envolveria uma modificação no banco de dados para incluir uma tabela de autores relacionada aos resumos, além de alterações no método de inscrição em turmas, permitindo a vinculação por meio de um código de resumo já criado ou por meio de convites enviados a outros alunos.

Outra melhoria seria a inclusão de uma forma mais estruturada para cadastrar as referências bibliográficas nos resumos, por meio de importação de referencias no padrão bibtex, em vez de utilizar apenas um campo de texto livre. Outra limitação encontrada foi a geração do sumário do caderno, pois o pacote LaTeX utilizado impedia a compilação completa do documento, e não foi possível identificar uma solução em tempo hábil.

Em suma, este trabalho compõe uma ferramenta estratégica para a centralização e divulgação dos indicadores dos projetos PEP ICT. A investigação de sistemas correlatos evidenciou que as tecnologias atuais não atendem plenamente às necessidades peculiares do campus do ICT-UNIFESP. Portanto, a plataforma desenvolvida preenche essa lacuna de maneira customizada, deixando uma base sólida e estruturado para tornar a extensão universitária mais organizada, eficiente e visível para a comunidade acadêmica.

6 Conclusão

Para este projeto, espera-se o desenvolvimento de um sistema que forneça as soluções propostas nos requisitos, a começar por uma interface intuitiva e organizada para que os alunos possam enviar os resultados de seus projetos PEP ICT e obter retornos dos professores. Para o corpo docente, a expectativa é a de poder criar turmas conforme suas disciplinas, analisar os resumos de forma isolada por projeto e fornecer avaliações acompanhadas de feedbacks construtivos. Por fim, para o coordenador, o sistema deveria permitir o gerenciamento de acessos, a visualização dos trabalhos aprovados e a seleção daqueles que integrariam o caderno de resumos, viabilizando a geração automatizada do caderno de resumos de maneira simplificada.

O projeto cumpre com êxito o seu objetivo geral ao entregar um repositório centralizado para a gestão dos programas PEP ICT, focando estritamente nas demandas específicas do campus do ICT. A solução desenvolvida apoia diretamente toda a comunidade acadêmica ao transformar um processo antes descentralizado e manual em um fluxo digital ágil, transparente e imune a erros operacionais comuns. Ao viabilizar a publicação semestral e automatizada das ações de extensão, o sistema não apenas simplifica o cotidiano de alunos, professores e coordenadores, mas também potencializa a visibilidade da produção científica e social da UNIFESP frente à comunidade externa.

A principal limitação enfrentada no desenvolvimento deste projeto residiu no cronograma restrito, o que exigiu escolhas estratégicas para priorizar as funcionalidades essenciais do sistema, tais como o fluxo de submissão de resumos e a compilação automatizada dos arquivos PDF em $\text{\LaTeX}$. Como consequência direta desse direcionamento, o *front-end* foi construído de maneira simplificada, utilizando apenas HTML nativo integrado ao *framework* Django. Embora a ideia inicial previsse uma interface gráfica mais refinada e estilizada, não houve tempo hábil para tal implementação, resultando em telas puramente funcionais. Da mesma forma, o banco de dados SQLite cumpriu com êxito o papel de viabilizar o protótipo dentro do prazo disponível, mas reconhece-se que tanto a persistência de dados quanto a interface visual podem ser revistas em etapas futuras, as quais incluirão, primordialmente, a aplicação de testes formais e a validação do sistema junto aos usuários potenciais reais.

Assim, abre-se caminho para a evolução da plataforma, sugerindo-se a migração para um banco de dados de produção (como PostgreSQL ou MySQL) e o desacoplamento da camada visual para uma estrutura moderna, como o React ou Angular, com o objetivo de conferir maior robustez, escalabilidade e uma experiência de usuário aprimorada. Adicionalmente, propõe-se a criação da mecânica de múltiplos autores por resumo e a

inclusão de referências estruturadas para o resumo. Somam-se a isso algumas correções a serem feitas, como a geração do sumário no caderno de revistas, que não está funcionando por enquanto, e um modelo padrão mais adequado para o caderno de resumos.

De maneira geral, esse trabalho traz uma base estruturada para uma plataforma que centralize e padronize a coleta e a divulgação das ações extensionistas curricularizadas nas unidades curriculares da graduação e pós-graduação no ICT-UNIFESP, auxiliando na melhoria desses projetos e aumentando seu alcance.

Referências

- BECK, K. et al. *Princípios do Manifesto para Desenvolvimento Ágil de Software*. 2001. Manifesto Agile. Disponível em: <<https://agilemanifesto.org/iso/ptbr/principles.html>>. Acesso em: 01 jun. 2026. Citado na página 24.
- BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. *UML: Guia do Usuário*. Rio de Janeiro: Elsevier Brasil, 2006. Disponível em: <<https://books.google.com.br/books?id=PZ0vOqf8f1sC>>. Acesso em: 02 jul. 2026. Citado 3 vezes nas páginas 31, 32 e 33.
- Django Software Foundation. *Class-based views | Django Documentation (Version 6.0)*. 2026. Disponível em: <<https://docs.djangoproject.com/en/6.0/topics/class-based-views/>>. Acesso em: 04 jul. 2026. Citado na página 22.
- Django Software Foundation. *Django at a glance | Django Documentation (Version 6.0)*. 2026. Disponível em: <<https://docs.djangoproject.com/en/6.0/intro/overview/>>. Acesso em: 04 jul. 2026. Citado na página 21.
- Django Software Foundation. *Models | Django Documentation (Version 6.0)*. 2026. Disponível em: <<https://docs.djangoproject.com/en/6.0/topics/db/models/>>. Acesso em: 04 jul. 2026. Citado na página 21.
- Django Software Foundation. *Templates | Django Documentation (Version 6.0)*. 2026. Disponível em: <<https://docs.djangoproject.com/en/6.0/topics/templates/>>. Acesso em: 04 jul. 2026. Citado na página 22.
- EasyChair. *EasyChair: Conference Management System*. 2026. Disponível em: <<https://easychair.org/>>. Acesso em: 16 jun. 2026. Citado na página 18.
- EDUCAÇÃO, M. D. *Lei nº 13.005, de 25 de junho de 2014. Aprova o Plano Nacional de Educação - PNE e dá outras providências*. Brasília, DF: [s.n.], 2014. Disponível em: <http://www.planalto.gov.br/ccivil_03/_ato2011-2014/2014/lei/l13005.htm>. Acesso em: 28 abr. 2026. Citado na página 16.
- LATEX PROJECT. *LaTeX: A Document Preparation System*. 2024. Disponível em: <<https://www.latex-project.org/>>. Acesso em: 23 jul. 2025. Citado na página 21.
- MBELLO. *pdflatex: Simple wrapper to calling pdflatex*. 2019. Disponível em: <<https://pypi.org/project/pdflatex/>>. Acesso em: 21 jun. 2026. Citado na página 26.
- MDN CONTRIBUTORS. *HTML: Linguagem de Marcação de Hipertexto*. 2026. Disponível em: <<https://developer.mozilla.org/pt-BR/docs/Web/HTML>>. Acesso em: 02 maio 2026. Citado na página 22.
- NACAGUMA, S.; STOCO, S.; ASSUMPÇÃO, R. P. S. (Ed.). *Política de curricularização da extensão na UNIFESP: caminhos, desafios e construções*. 1. ed. São Paulo: Alameda, 2021. Recurso eletrônico. ISBN 978-65-5966-078-0. Disponível em: <<https://proreitoria.unifesp.br/proec/a-proec/curricularizacao/e-book-pdf-politica-de-curricularizacao-da-extensao-na-unifesp>>. Acesso em: 04 jul. 2026. Citado 2 vezes nas páginas 16 e 20.

Open Library of Humanities. *Janeway: Open Source Journal Publishing Platform*. 2026. Disponível em: <<https://janeway.systems/>>. Acesso em: 16 jun. 2026. Citado na página 19.

OWENS, M.; ALLEN, G. *The Definitive Guide to SQLite*. 2nd. ed. New York: Apress, 2010. Disponível em: <<https://link.springer.com/book/10.1007/978-1-4302-3226-1>>. Acesso em: 02 maio 2026. Citado na página 23.

Public Knowledge Project. *Open Journal Systems (OJS)*. 2026. Disponível em: <<https://pkp.sfu.ca/software/ojs/>>. Acesso em: 16 jun. 2026. Citado na página 18.

Public Knowledge Project. *PKP History Timeline*. 2026. Disponível em: <<https://pkp.sfu.ca/about/timeline/>>. Acesso em: 16 jun. 2026. Citado na página 18.

Python Packaging Authority. *Pipenv: Python Development Workflow for Humans*. 2026. Disponível em: <<https://pipenv.pypa.io/en/latest/>>. Acesso em: 21 jun. 2026. Citado na página 26.

RED HAT. *O que é a metodologia Agile?* 2024. Website oficial da Red Hat. Disponível em: <<https://www.redhat.com/pt-br/topics/devops/what-is-agile-methodology>>. Acesso em: 01 jun. 2026. Citado na página 25.

Sociedade Brasileira de Computação. *JEMS – Journal and Event Management System*. 2026. Disponível em: <<https://jems.sbc.org.br/jems2/index.php?r=site/login>>. Acesso em: 19 jun. 2026. Citado na página 19.

Sociedade Brasileira de Computação. *Manual do JEMS – Visão geral do JEMS*. 2026. Disponível em: <https://jems.sbc.org.br/manual/index.php?page=Visao_geral_do_JEMS>. Acesso em: 19 jun. 2026. Citado na página 19.

Sociedade Brasileira de Computação. *Tabela de Serviços JEMS: Valores de Serviços do JEMS para Eventos Sem Apoio da SBC*. 2026. Atualização: janeiro/2026. Disponível em: <<https://www.sbc.org.br/wp-content/uploads/2025/10/Tabelas-JEMS-Sem-apoio-2026.pdf>>. Acesso em: 28 jun. 2026. Citado na página 19.

SQLite CONSORSIUM. *About SQLite*. 2025. Disponível em: <<https://sqlite.org/about.html>>. Acesso em: 02 maio 2026. Citado 2 vezes nas páginas 22 e 23.

SQLite Consortium. *Appropriate Uses for SQLite*. 2026. Documentação Oficial do SQLite. Disponível em: <<https://sqlite.org/whentouse.html>>. Acesso em: 29 jun. 2026. Citado na página 60.

TOKUMOTO, A. F. S. *Plataforma de resumos estruturados*. GitHub, 2026. Disponível em: <<https://github.com/andretokumoto/Plataforma-de-resumos-estruturados>>. Acesso em: 28 jun. 2026. Citado na página 36.

TOKUMOTO, A. F. S. *Plataforma de Resumos (Projeto de Trabalho de Conclusão de Curso)*. 2026. Acessado em: 05 jul. 2026. Disponível em: <<https://plataforma-resumos.fly.dev/>>. Citado na página 59.

UFF. *Apostila de LaTeX*. [S.l.], 2004. Apostila. Disponível em: <<https://wp.ufpel.edu.br/fernandosimoes/files/2012/06/latex-introducao.pdf>>. Acesso em: 22 out. 2025. Citado na página 21.

WAKODE, R. B.; RAUT, L. P.; TALMALE, P. Overview on kanban methodology and its implementation. *IJSRD - International Journal for Scientific Research & Development*, v. 3, n. 02, p. 2518–2521, 2015. Disponível em: <<https://www.ijssrd.com/articles/IJSRDV3I2771.pdf>>. Acesso em: 10 maio 2026. Citado na página 25.